

# DeepSeek-Inspired Efficiency Under Data-Limited Training: A Controlled Study of MLA, Sparse MoE Routing, V3-Style Load Balancing, and Multi-Token Prediction

Marko Miovski

September 17, 2026

## Abstract

Architectural efficiency mechanisms can improve predictive quality or reduce analytical per-token computation, but their realized value depends on the resource being measured and the systems support available. This study evaluates six DeepSeek-inspired decoder variants—a dense control, a Multi-Head Latent Attention (MLA) analogue, multi-token prediction (MTP), sparse mixture-of-experts (MoE), MLA+MoE, and V3-style expert-bias routing—under one local protocol. Each model was trained at nominal budgets of 10M, 25M, and 50M tokens with ten aligned seeds, yielding 180 completed runs. Every run used adjacent-pair rotary embeddings and the same 400 fixed, non-overlapping evaluation windows per held-out split. Test next-token cross-entropy was the primary inferential outcome, analyzed with repeated-measures global tests, seven planned paired contrasts, exact sign-flip tests, percentile-bootstrap intervals, and the Holm adjustment across 21 comparisons. Efficiency was measured separately through synchronized training-step throughput, active end-to-end throughput, training and evaluation peak GPU allocation, and total-parameter versus activated-parameter exposure. All six models improved from 10M to 50M, with mean test loss decreasing by 0.592–0.634 and every aligned seed improving. MoE achieved the lowest mean test loss at each budget and improved on Dense under the primary adjustment at all three budgets, including a 0.027 reduction at 50M. It activated 35.7% of its 220.146 million stored parameters but was approximately 48% slower in training-step throughput and used 50.7% more peak training allocation than Dense. MTP established a smaller test-loss advantage at 50M while imposing a 13.5% throughput cost and 12.9% memory cost. MLA remained worse than Dense in quality, throughput, and memory; adding MoE improved MLA but did not match ordinary MoE. V3-style routing did not improve quality, throughput, memory, or terminal load balance over the auxiliary-loss MoE baseline. The evidence therefore supports no universal winner: MoE provided the strongest predictive quality and activated-path exposure, Dense provided the strongest local training systems profile, and MTP provided a budget-specific middle tradeoff.

## Contents

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                      | <b>4</b>  |
| 1.1      | Motivation: Architectural Efficiency Under Constrained Compute . . . . . | 4         |
| 1.2      | Study Problem and Scope . . . . .                                        | 5         |
| 1.3      | Research Questions . . . . .                                             | 5         |
| 1.4      | Contributions . . . . .                                                  | 6         |
| <b>2</b> | <b>Background and Related Work</b>                                       | <b>6</b>  |
| 2.1      | Dense Decoder-Only Transformers as the Control . . . . .                 | 6         |
| 2.2      | Multi-Head Latent Attention . . . . .                                    | 7         |
| 2.3      | Sparse Mixture-of-Experts Models . . . . .                               | 7         |
| 2.4      | Routing Balance and V3-Style Expert-Bias Control . . . . .               | 8         |
| 2.5      | Multi-Token Prediction . . . . .                                         | 8         |
| 2.6      | Data-Limited and Overparameterized Training . . . . .                    | 9         |
| <b>3</b> | <b>Methods and Experimental Design</b>                                   | <b>10</b> |
| 3.1      | Study Design and Experiment Matrix . . . . .                             | 10        |
| 3.2      | Data, Tokenizer, and Sequence Construction . . . . .                     | 11        |
| 3.3      | Controlled Model Family . . . . .                                        | 11        |
| 3.4      | Training Objectives and Optimization Protocol . . . . .                  | 13        |
| 3.5      | Evaluation Metrics and Parameter Exposure . . . . .                      | 14        |
| 3.6      | Statistical Analysis . . . . .                                           | 15        |
| 3.7      | Hardware, Reproducibility, and Implementation Boundaries . . . . .       | 16        |
| <b>4</b> | <b>Results and Discussion</b>                                            | <b>17</b> |
| 4.1      | Overall Quality and Token-Budget Scaling . . . . .                       | 17        |
| 4.2      | MLA and Sparse-Capacity Tradeoffs . . . . .                              | 19        |
| 4.3      | Interaction Between MLA and MoE . . . . .                                | 20        |
| 4.4      | V3-Style Routing Behavior . . . . .                                      | 21        |
| 4.5      | Multi-Token Prediction . . . . .                                         | 23        |
| 4.6      | Cross-Mechanism Quality, Systems Cost, and Exposure Synthesis . . . . .  | 24        |
| <b>5</b> | <b>Limitations and Threats to Validity</b>                               | <b>27</b> |
| 5.1      | Scale and Implementation Fidelity . . . . .                              | 27        |
| 5.2      | Measurement and Evaluation Design . . . . .                              | 27        |
| 5.3      | Generalization and Untested Claims . . . . .                             | 28        |

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>6 Conclusion</b>                                  | <b>29</b> |
| 6.1 Main Findings . . . . .                          | 29        |
| 6.2 Implications and Future Work . . . . .           | 30        |
| <b>A Full Descriptive Results</b>                    | <b>32</b> |
| <b>B Complete Inferential Results</b>                | <b>38</b> |
| <b>C Mechanism Diagnostics</b>                       | <b>42</b> |
| <b>D Reproducibility and Artifact Lineage</b>        | <b>43</b> |
| D.1 Canonical artifact chain . . . . .               | 43        |
| D.2 Configuration and execution boundaries . . . . . | 43        |
| D.3 Regeneration and audit scope . . . . .           | 44        |
| D.4 Presentation policy . . . . .                    | 44        |

## 1. Introduction

Large language models have improved as model capacity, training data, and computation have increased but those gains have made efficiency a central design problem. A larger dense Transformer stores more parameters and activates the same feed-forward path for every token. This can improve representational capacity, yet it also increases memory use and computation unless the model is trained and served with sufficient data and hardware [9, 11]. Due to this, recent architectures seek to separate stored capacity from per-token work, compress attention state, control sparse routing, or provide denser supervision during pretraining. DeepSeekMoE, DeepSeek-V2, and DeepSeek-V3 combine several of these ideas at production scale. This includes fine-grained sparse experts, shared experts, Multi-Head Latent Attention (MLA), auxiliary-loss-free load balancing, and multi-token prediction (MTP) [2–4].

These mechanisms are promising but their reported advantages do not transfer automatically to smaller systems. Production gains may depend on long-context inference, large batches, expert parallelism, fused kernels, communication libraries, or training budgets large enough for specialization to emerge. A mechanism can reduce an analytical quantity such as activated parameter count while still increasing wall-clock time or device memory in a local implementation. Similarly, an auxiliary objective can be learned successfully without improving ordinary next-token prediction. This means evaluating such mechanisms under constrained hardware requires more than reproducing their names or reporting a single validation loss. It requires a controlled comparison that measures predictive quality, systems cost, parameter exposure, and mechanism-specific behavior separately.

### 1.1 Motivation: Architectural Efficiency Under Constrained Compute

Efficiency in language modeling is multidimensional. Predictive quality matters but so do the number of parameters that must be stored, the number activated for each token, the memory allocated during training, the rate at which tokens are processed, and the stability of any routing mechanism. These quantities need not move together. Sparse MoE models, for example, can expose each token to only a fraction of total model capacity, yet the complete expert bank must still reside in memory and the routing path introduces dispatch and control overhead [7, 12]. MLA is motivated primarily by reducing the key–value cache during autoregressive inference but its projection and reconstruction operations may remain costly during full-sequence training [3]. MTP supplies additional future-token targets but the extra optimization work does not guarantee lower next-token loss at every model scale or token budget [8].

The amount of training data relative to model capacity further changes how these mechanisms should be interpreted. Scaling-law studies show that parameters and tokens must be considered jointly. Increasing model size without a corresponding increase in training data can leave capacity underexposed [9, 11]. This issue is particularly important for sparse models because total parameters and activated parameters describe different aspects of the architecture. Tokens per total parameter characterize exposure of stored capacity, whereas tokens per activated parameter characterize approximate exposure of the computation used on each token path. Because of this, a local study should report both views rather than treating parameter count as a single undifferentiated quantity.

This study is motivated by the need for that controlled, mechanism-aware evidence. The objective is not to identify the largest or most capable language model that can be trained locally. It is to determine what happens when several DeepSeek-inspired mechanisms are implemented in a common decoder framework, trained under the same short-to-medium token budgets, and evaluated with aligned

experimental units. The constrained setting is useful precisely because it exposes which benefits appear directly from the architecture and which appear to depend on larger-scale systems support.

## 1.2 Study Problem and Scope

The study compares six decoder-only model families: a dense control, an MLA analogue, an MTP model, a sparse MoE model, a combined MLA+MoE model, and a V3-style routing model. Each family was trained at nominal budgets of 10M, 25M, and 50M tokens using ten aligned seeds, producing a balanced matrix of 180 completed runs. All runs used the same FineWeb-Edu-derived corpus sample, tokenizer, context length, optimizer family, fixed held-out windows, and local hardware environment. Test next-token cross-entropy is the primary quality outcome, supplemented by validation loss, perplexity, synchronized training-step throughput, active end-to-end throughput, separate training and evaluation peak GPU allocation, parameter exposure, and terminal routing or MTP diagnostics.

The comparison is intentionally mechanism-focused rather than parameter-matched in every dimension. The dense, MLA, and MTP models differ in total parameter count while the sparse models contain substantially more total capacity but activate a smaller subset per token. The study therefore does not ask which architecture is universally best at a fixed parameter count. It asks how the implemented mechanisms change the joint tradeoff among quality, storage capacity, activated capacity, training cost, and stability under one controlled protocol.

The scope is also narrower than a full DeepSeek reproduction. The implementation excludes distributed expert parallelism, production all-to-all communication, custom sparse kernels, FlashMLA, generation-time KV-cache benchmarking, speculative decoding, instruction tuning, reinforcement learning, and reasoning evaluation. The results consequently describe the local implementations, not the complete DeepSeek-V2, DeepSeek-V3, or DeepSeek-R1 systems [5]. This boundary is essential because a negative local training result does not invalidate a mechanism designed for optimized inference or distributed execution, and a positive local result would not establish frontier-scale superiority.

## 1.3 Research Questions

The central research question is:

*How do DeepSeek-inspired architectural and training-efficiency techniques affect model quality, memory usage, throughput, activated parameter count, expert utilization, routing stability, and optimization behavior when implemented at small scale under fixed compute constraints?*

The controlled architecture ladder gives this question the six original mechanism-specific forms:

1. What is the control model’s quality and efficiency?
2. Does MLA-style compression improve memory or throughput at small scale?
3. Does sparse routing improve activated-parameter efficiency?
4. Do the V2-style components remain stable together?
5. Does routing design improve utilization and stability?

## 6. Does MTP improve optimization or sample efficiency?

A cross-cutting question concerns the data-limited regime: how do the conclusions change when sparse models are viewed through tokens per total parameter rather than tokens per activated parameter? This distinction is used to interpret capacity and computation rather than to claim a new scaling law. The experiment also does not provide the same-architecture capacity sweep required to identify classical double descent.

### 1.4 Contributions

This study makes four contributions. First, it provides a balanced, multi-seed comparison of six related model families across three token budgets. The use of ten aligned seeds permits paired comparisons and avoids treating the 180 runs as independent observations. The design is broad enough to compare the major mechanisms while retaining a common training and evaluation protocol.

Second, the study evaluates efficiency through several distinct quantities rather than through predictive loss or parameter count alone. It reports total, trainable, and activated parameters; token exposure under each definition; synchronized training-step and active end-to-end throughput; separate training and evaluation peak GPU allocation; and mechanism-specific routing and MTP diagnostics. This separates theoretical sparse-capacity advantages from realized local systems behavior.

Third, the analysis uses a prespecified paired statistical framework. Test loss is treated as the primary inferential outcome, with repeated-measures global tests, exact paired sign-flip tests, paired bootstrap intervals, standardized paired effects, and the Holm adjustment across the complete 21-contrast test-loss family.

Fourth, the study provides a mechanism-first interpretation of the results. It connects observed differences to the architectural operations that could plausibly produce them while distinguishing supported explanations from causal proof. This is important because the main empirical lesson is not that one mechanism dominates every metric. Rather, the mechanisms redistribute capacity, optimization signal, memory demand, and execution overhead in different ways. Additionally, their value depends on which resource constraint is being optimized. The resulting evidence is intended to clarify those tradeoffs within a constrained, data-limited training regime.

## 2. Background and Related Work

The mechanisms studied here come from a broader effort to increase language-model capacity without allowing training and inference cost to grow at the same rate. Prior work has approached that problem through attention compression, conditional computation, routing control, and denser prediction objectives. These mechanisms were originally developed and validated in substantially larger systems, often alongside specialized kernels, distributed execution, and long training runs. The purpose of this section is to explain the intended role of each mechanism and the tradeoff it is designed to address.

### 2.1 Dense Decoder-Only Transformers as the Control

The Transformer replaced recurrence with self-attention and position-wise feed-forward layers, allowing each token representation to combine information from the preceding context through learned attention weights [17]. Decoder-only language models apply this architecture autoregressively: given tokens  $x_{<t}$ ,

the model estimates  $p_\theta(x_t \mid x_{<t})$  and is trained by next-token cross-entropy. This objective became the standard pretraining formulation for modern generative language models [1].

In a dense Transformer, the same attention and feed-forward parameters are available to every token. This means total, trainable, and per-token activated parameter counts coincide when all parameters are trainable. Dense execution is comparatively simple because it maps well to large matrix multiplications, avoids routing decisions, and provides a clear relationship between stored capacity and work performed for each token. It is consequently the appropriate control for determining whether a more specialized mechanism improves quality, memory use, throughput, or parameter exposure rather than merely adding architectural complexity.

Prior scaling studies show that language-model loss depends jointly on model size, training data, and compute rather than on parameter count alone [9, 11]. A larger model can be more sample-efficient but it can also remain undertrained when the token budget does not grow with capacity. This interaction motivates holding the training-token budget fixed while reporting both predictive quality and systems cost.

## 2.2 Multi-Head Latent Attention

Standard multi-head attention forms separate key and value representations for each layer and attention head. During autoregressive generation, previously computed keys and values are retained in a key-value (KV) cache so that the model does not recompute the full prefix at every decoding step. The cache grows with sequence length, layer count, and key/value dimensionality, thereby making memory capacity and memory bandwidth important inference constraints.

DeepSeek-V2 introduced Multi-Head Latent Attention (MLA) to reduce this cost [3]. Rather than storing full head-specific key and value states, MLA compresses them into a lower-dimensional latent representation from which the required attention states can be recovered. The central efficiency claim is consequently tied to KV-cache compression during autoregressive inference. It does not imply that every MLA implementation must reduce full-sequence training memory or improve training throughput; additional projection and reconstruction operations can remain visible during training, especially without fused kernels or a cache-aware decoding path.

The current study implements an MLA-style latent projection within the same decoder framework as the dense control. This preserves the main architectural idea—compressing the attention representation before reconstructing head-specific states—while treating production KV-cache management, FlashMLA, and kernel-level optimization as out of scope. The relevant empirical question is correspondingly narrow and asks whether this local analogue improves measured training quality or systems behavior under the selected scale and hardware.

## 2.3 Sparse Mixture-of-Experts Models

Mixture-of-Experts (MoE) models increase total capacity through conditional computation. In a Transformer MoE layer, a router assigns each token to a small subset of feed-forward experts and only those selected experts process that token. This indicates the model can store substantially more parameters than it activates on a single token path. GShard and Switch Transformers demonstrated that this separation can support very large models while also identifying routing imbalance, communication cost, and training instability as central engineering problems [7, 12].

DeepSeekMoE extends this design with two ideas intended to improve expert specialization: finer

expert segmentation and the isolation of shared experts [2]. Finer segmentation permits a token to combine a more flexible set of narrower routed experts while shared experts capture information that may otherwise be redundantly learned across routed experts. DeepSeek-V2 combines this sparse feed-forward design with MLA, framing the model in terms of a large total parameter count but a much smaller number of parameters activated for each token [3].

This distinction is essential but incomplete. Fewer activated parameters do not automatically imply lower wall-clock time or lower device memory. Expert weights must still be stored, routing and token dispatch introduce overhead, and efficient sparse execution may depend on large batches, expert parallelism, communication primitives, and specialized kernels. For this reason, the study treats total capacity, activated capacity, throughput, and peak memory as separate measurements rather than using activated parameter count as a proxy for realized speed.

## 2.4 Routing Balance and V3-Style Expert-Bias Control

Sparse models require the router to distribute tokens across experts without allowing a small subset to absorb most assignments. Conventional MoE systems commonly add an auxiliary load-balancing term to the language-model objective. This supplies direct gradient pressure toward more even routing but it also changes the optimization target because a sufficiently strong balancing term can compete with the representations preferred by next-token prediction.

DeepSeek-V3 introduced an auxiliary-loss-free balancing strategy intended to separate load control from the main model objective [4]. Instead of adding the principal balancing signal to the loss, the router maintains expert-specific selection biases that are adjusted according to observed expert load. This implies the logits used to choose experts are shifted toward underused experts and away from overused experts while the original affinity scores continue to determine the routing weights.

The design creates a direct comparison between two forms of control. Auxiliary-loss routing modifies optimization through backpropagation; expert-bias routing modifies discrete expert selection through an external update rule. Their behavior cannot be judged from language-model loss alone. Routing entropy describes how dispersed the router probabilities are while expert-load variance describes how evenly top- $k$  selections are distributed. Neither statistic independently proves useful specialization because concentrated routing may indicate either meaningful specialization or collapse, and uniform routing may reflect balance without differentiated expert function. For that reason, the study evaluates quality and terminal routing behavior together.

## 2.5 Multi-Token Prediction

Standard autoregressive pretraining supervises one next-token target at each position. Multi-token prediction (MTP) augments that objective by asking a shared model representation to predict several future tokens through separate prediction heads. The motivation is to provide a denser learning signal and encourage representations that remain useful beyond the immediately following token. Prior work reports that the benefit can increase with model scale and can be especially pronounced on generative tasks such as code while also creating opportunities for faster inference when multiple proposed tokens are verified together [8].

DeepSeek-V3 includes MTP as a pretraining objective alongside its sparse architecture and routing changes [4]. Those large-scale results do not establish that the same objective will improve a smaller model trained for a short token budget. Auxiliary heads add parameters and optimization work, and

success on their future-token losses need not transfer to lower ordinary next-token loss.

The local implementation uses two auxiliary future-token heads on top of the shared decoder trunk. The evaluation remains next-token cross-entropy, which permits a controlled comparison with the dense baseline. Generation-time verification, speculative decoding, instruction tuning, and reasoning evaluation are excluded. This separation is important because an MTP model may learn its auxiliary task without demonstrating either better next-token quality or faster decoding in the tested system.

## 2.6 Data-Limited and Overparameterized Training

Empirical scaling laws describe language-model loss as a function of parameters, data, and compute [11]. Later compute-optimal analysis showed that increasing parameter count without a corresponding increase in training tokens can leave large models undertrained, and that model size and data should be scaled together when compute is allocated optimally [9]. These findings make the capacity-to-data ratio central to any comparison performed under fixed token budgets.

The interpretation is more complicated for sparse models. Total parameters describe stored representational capacity while activated parameters describe the subset used on a token path. A sparse model may receive relatively few tokens per total parameter but more tokens per activated parameter than a smaller dense model. Trainable parameters form a third concept when some stored parameters are frozen, although all parameters were trainable in the present experiment. Reporting these quantities separately avoids treating a single parameter count as a complete measure of either optimization exposure or computational work.

The term *overparameterized* is used here in this capacity-to-data sense, not as a direct claim about a classical interpolation threshold. Language-model tokens are dependent, contexts overlap, and the effective number of examples is not equivalent to the raw token count. Sparse routing further prevents a simple one-to-one analogy between total parameters and the parameters updated by each token. Likewise, classical double descent cannot be established without a sufficiently dense same-architecture capacity sweep.

Most prior evidence for MLA, DeepSeekMoE, auxiliary-loss-free routing, and MTP comes from large systems in which several mechanisms and systems optimizations operate together [3, 4]. That evidence demonstrates feasibility at scale but makes it difficult to determine how each component behaves under constrained local training. This study addresses that narrower gap by introducing the mechanisms progressively, holding the corpus, tokenizer, context length, optimizer, token budgets, seed set, and evaluation protocol fixed while also measuring quality alongside throughput, memory, parameter exposure, and routing diagnostics.

### 3. Methods and Experimental Design

#### 3.1 Study Design and Experiment Matrix

**Table 1:** Canonical experimental protocol. The design was balanced across six models, three token budgets, and ten aligned seeds. Actual token counts exceed the nominal budgets by at most 960 tokens because each optimizer step processes 1,024 targets.

| Component               | Canonical setting                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Experiment matrix       | Six models $\times$ three budgets $\times$ ten aligned seeds = 180 completed runs                                         |
| Token budgets           | 10M, 25M, and 50M nominal training target tokens                                                                          |
| Seeds                   | 1337, 2027, 4441, 5501, 6173, 8191, 10007, 11213, 12721, 31415                                                            |
| Corpus                  | FineWeb-Edu <b>sample-10BT</b> at pinned revision; 50,000 train, 1,000 validation, and 1,000 test documents               |
| Tokenized data          | 58,814,038 train; 1,067,535 validation; 1,159,754 test tokens                                                             |
| Tokenizer               | Byte-level BPE, vocabulary 10,000, minimum frequency 2, trained only on retained training text                            |
| Common model geometry   | 12 layers, 12 heads, width 768, context 256, dropout 0.1, RMSNorm, adjacent-pair RoPE, tied embeddings                    |
| Optimization            | AdamW; learning rate $3 \times 10^{-4}$ ; weight decay 0.1; betas (0.9, 0.95); gradient clipping 1.0; fixed learning rate |
| Batch and precision     | Four sequences $\times$ 256 targets = 1,024 target tokens per step; fp32; deterministic algorithms                        |
| Evaluation              | 400 fixed, corpus-spanning, non-overlapping windows per held-out split; 102,400 scored tokens per split at every budget   |
| Hardware and software   | NVIDIA GeForce RTX 4050 Laptop GPU; Python 3.11.9; PyTorch 2.6.0+cu124; CUDA 12.4                                         |
| <b>Budget execution</b> | <b>Steps / actual tokens / overshoot / evaluation batches</b>                                                             |
| 10M                     | 9,766 / 10,000,384 / 384 (0.00384%) / 100                                                                                 |
| 25M                     | 24,415 / 25,000,960 / 960 (0.00384%) / 100                                                                                |
| 50M                     | 48,829 / 50,000,896 / 896 (0.00179%) / 100                                                                                |

The seed is the repeated experimental unit. Training windows were randomly sampled with replacement. The identical held-out window set was used for every model, seed, and budget.

The study used a balanced factorial design to compare six decoder-only language-model variants under the same local training environment. The model families were Dense, MLA, MTP, MoE, MLA+MoE, and V3 Routing. Each model was trained at nominal budgets of 10, 25, and 50 million target tokens using the same ten seeds: 1337, 2027, 4441, 5501, 6173, 8191, 10007, 11213, 12721, and 31415. The resulting matrix contained  $6 \times 3 \times 10 = 180$  completed runs.

The aligned seeds make the design paired. For a given budget, models with the same seed were initialized and trained under the same seed-controlled sampling conditions. The seed, rather than an individual token or evaluation window, is the repeated experimental unit. This pairing was retained in the omnibus tests, planned contrasts, confidence intervals for paired differences, and seed-level win counts. The design supports controlled comparisons among the implemented mechanisms but it does not isolate every architectural component independently because the sparse models necessarily differ from the dense models in both total capacity and feed-forward structure.

### 3.2 Data, Tokenizer, and Sequence Construction

The corpus was drawn from the `sample-10BT` configuration of FineWeb-Edu at pinned revision `87f09149ef4734204d70ed1d046ddc9ca3f2b8f9` [15]. A single streaming source split was shuffled with a buffer of 10,000 records and seed 1337. The shuffled stream was consumed sequentially to create disjoint local artifacts containing 1,000 validation documents, 1,000 test documents, and 50,000 training documents. Newlines were normalized, leading and trailing whitespace was removed, and empty records were discarded. Source-record manifests and SHA-256 hashes retain the exact split membership.

A byte-level byte-pair-encoding tokenizer was trained only on the retained training text, with a requested and realized vocabulary size of 10,000 and minimum token frequency of two. The complete byte alphabet was seeded, no training-text cap was applied, and encoding added no BOS, EOS, padding, or unknown tokens to the continuous language-model streams. Tokenization produced flat `int32` memory-mapped arrays containing 58,814,038 training tokens, 1,067,535 validation tokens, and 1,159,754 test tokens.

All models used a context length of 256. During training, each example consisted of 256 input tokens and the following 256 next-token targets. Start positions were sampled uniformly with replacement from the training memmap. Training windows could consequently overlap or repeat and the processed tokens should not be interpreted as independent observations.

Validation and test evaluation used the same fixed sample at every model, seed, and budget. Each split supplied 400 deterministic, non-overlapping 256-token windows distributed across the full held-out token array, for 102,400 scored target tokens per split. Each evaluation batch contained four windows, so every evaluation used 100 batches. The retained start-index hashes are `e2b4a5afcb9e` for validation and `5c513d7b7eba` for test. Because the held-out sample is identical across budgets, within-model budget changes are not confounded by budget-dependent evaluation prefixes.

### 3.3 Controlled Model Family

All six variants used the same decoder depth, width, vocabulary, context length, dropout, normalization, positional encoding, and tied input/output embeddings. The common backbone contained 12 decoder layers, 12 attention heads, model width 768, dropout 0.1, RMS normalization, rotary positional encoding [16], and tied token-embedding and language-model-head weights. The rotary implementation repeats one frequency across each adjacent even-odd feature pair before applying the paired rotation. Dense feed-forward blocks used a SwiGLU hidden width of 3,072. The experiment changed only the attention, feed-forward routing, or auxiliary-objective path required by each comparison.

**Table 2:** Controlled model family and parameter accounting. Activated parameters include all non-routed parameters, the shared experts, and the average top-2 routed-expert path in each MoE layer. They are an architecture-level exposure estimate, not measured FLOPs.

| Model      | Controlled mechanism                                                      | Total params. | Activated params. | Act./total |
|------------|---------------------------------------------------------------------------|---------------|-------------------|------------|
| Dense      | Dense attention and dense SwiGLU feed-forward blocks                      | 120,945,408   | 120,945,408       | 1.000      |
| MLA        | Local MLA attention analogue; dense SwiGLU feed-forward blocks            | 137,460,480   | 137,460,480       | 1.000      |
| MTP        | Dense backbone with separate offset-two and offset-three auxiliary heads  | 136,305,408   | 136,305,408       | 1.000      |
| MoE        | Dense attention; 12 routed and one shared expert per layer; top-2 routing | 220,146,432   | 78,588,672        | 0.357      |
| MLA+MoE    | Local MLA attention analogue combined with the auxiliary-loss MoE path    | 236,661,504   | 95,103,744        | 0.402      |
| V3 Routing | MoE geometry with auxiliary-loss-free expert-bias selection control       | 220,146,432   | 78,588,672        | 0.357      |

All parameters were trainable, so total and trainable parameter counts were identical. Dense feed-forward width was 3,072. Each routed or shared expert used width 512. MoE variants used 12 routed experts, one shared expert, normalized softmax top-2 weights, and no token dropping. MLA used key-value latent dimension 192, rotary-query dimension 64, non-positional query/key dimension 128 per head, and value dimension 128 per head. V3 Routing updated its non-gradient expert bias at rate 0.001 after each training forward and clipped it to  $[-1, 1]$ .

The Dense model served as the reference architecture. MLA replaced dense attention with a local Multi-Head Latent Attention analogue inspired by DeepSeek-V2 [3]. It used a 192-dimensional key-value latent projection, a 64-dimensional rotary query component, 128 non-positional query/key dimensions per head, and 128 value dimensions per head. This implementation was intended to test the local behavior of latent attention projections; it did not reproduce the optimized cache, kernel, or deployment stack of the original system.

MTP retained the dense backbone and added two separate auxiliary output heads. The heads predicted tokens at offsets two and three from the decoder inputs while the ordinary language-model head predicted the next token at offset one. This local design was motivated by multi-token prediction [8]; it did not implement the sequential MTP modules used at frontier scale, shared-head MTP, generation-time verification, or speculative decoding.

MoE replaced each dense feed-forward block with a local DeepSeekMoE-style layer [2]. Each layer contained 12 routed experts and one shared expert. The router selected two routed experts per token using softmax scores, normalized the selected weights, and never dropped tokens. Every expert used hidden width 512. MoE and MLA+MoE used a differentiable load-balancing auxiliary loss with weight 0.01. MLA+MoE combined the same MLA attention analogue with the same auxiliary-loss MoE feed-forward path.

V3 Routing used the MoE architecture and expert geometry but removed the differentiable routing auxiliary loss. Instead, it used a non-gradient expert-selection bias inspired by the auxiliary-loss-free

strategy described for DeepSeek-V3 [4]. After each training forward pass, the bias was updated toward uniform expert load at rate 0.001 and clipped to  $[-1, 1]$ . This controller affected expert selection but did not change the router probabilities used to weight selected expert outputs.

### 3.4 Training Objectives and Optimization Protocol

For a set of  $N$  target tokens with contexts  $c_j$  and targets  $y_j$ , ordinary next-token cross-entropy was

$$\hat{L}_{\text{NTP}} = -\frac{1}{N} \sum_{j=1}^N \log p_{\theta}(y_j | c_j). \quad (1)$$

Dense and MLA optimized only this objective. For MoE and MLA+MoE, the training loss also included the sum of the 12 layer-level load-balancing losses,

$$L_{\text{train}} = L_{\text{NTP}} + \sum_{\ell=1}^{12} L_{\text{aux},\ell}, \quad L_{\text{aux},\ell} = \lambda_{\text{aux}} E \sum_{e=1}^E f_{\ell e} \bar{p}_{\ell e}, \quad (2)$$

where  $E = 12$  routed experts,  $f_{\ell e}$  is expert  $e$ 's fraction of top- $k$  selections in layer  $\ell$ ,  $\bar{p}_{\ell e}$  is its mean router probability, and  $\lambda_{\text{aux}} = 0.01$ . V3 Routing set this auxiliary term to zero and adjusted its expert-selection biases outside gradient descent.

The MTP model optimized

$$L_{\text{train}} = L_{\text{NTP}} + 0.3 \left( \frac{L_2 + L_3}{2} \right), \quad (3)$$

where  $L_2$  and  $L_3$  are the cross-entropies from the separate offset-two and offset-three auxiliary heads. Because the stored training loss combines different terms for ordinary, MoE, and MTP models, combined training loss is not used for cross-model quality comparisons. Validation and test next-token loss from Equation 1 provide the common quality measure.

All runs used AdamW [13] with learning rate  $3 \times 10^{-4}$ , weight decay 0.1, momentum coefficients (0.9, 0.95), gradient-norm clipping at 1.0, and no learning-rate schedule. Training used full 32-bit floating-point precision with deterministic algorithms enabled. Each step processed four sequences of 256 target tokens, or 1,024 target tokens per optimizer update. The 10M, 25M, and 50M budgets required 9,766, 24,415, and 48,829 steps and terminated after 10,000,384, 25,000,960, and 50,000,896 observed target tokens. The resulting overshoots were 384, 960, and 896 tokens, all below 0.004% of the requested budget.

Periodic validation and atomic checkpoints occurred every 1,000, 2,500, or 5,000 steps for the 10M, 25M, or 50M protocol. Logging occurred every 100, 250, or 500 steps, respectively. Each run concluded with final validation and test evaluation. Interrupted runs could resume only after the code tree, configuration, data, tokenizer, environment, and experiment identity matched the checkpoint; model, optimizer, random-number-generator, and random-window sampler state were restored. Completed summaries were then semantically validated and treated as immutable.

### 3.5 Evaluation Metrics and Parameter Exposure

#### 3.5.1 Predictive Quality

Validation and test loss were computed as target-token-weighted means of next-token cross-entropy over the fixed 400-window samples. Perplexity was derived as

$$\text{PPL} = e^{\hat{L}_{\text{NTP}}}. \quad (4)$$

Lower values indicate better next-token prediction. Test loss was designated as the primary inferential outcome. Perplexity is a monotonic transformation of loss and was retained for interpretation rather than treated as an independent primary endpoint.

#### 3.5.2 Parameter Accounting and Exposure

The study separated total, trainable, and activated parameters. Total parameters represent stored model capacity. Trainable parameters represent the parameters exposed to optimization. Activated parameters approximate the parameters used for one token’s forward path: all non-routed parameters plus the shared experts and the average top- $k$  routed experts in each MoE layer. This analytical quantity is not a measurement of floating-point operations.

For a run processing  $T$  training target tokens, the three exposure ratios were

$$E_{\text{total}} = \frac{T}{P_{\text{total}}}, \quad E_{\text{trainable}} = \frac{T}{P_{\text{trainable}}}, \quad E_{\text{activated}} = \frac{T}{P_{\text{activated}}}. \quad (5)$$

They are interpreted as storage/capacity exposure, optimization exposure, and approximate per-token compute exposure, respectively. All parameters required gradients in the run matrix, so total and trainable exposure were numerically identical. Activated exposure remained distinct for the sparse models because only two routed experts and the shared expert were active per token in each MoE layer.

#### 3.5.3 Systems Metrics

The primary training-step throughput metric was

$$v_{\text{step}} = \frac{T_{\text{train}}}{\sum_s \Delta t_s}, \quad (6)$$

where each synchronized step interval included batch transfer, forward and backward computation, gradient clipping, and the optimizer update. It excluded periodic evaluation, logging callbacks, checkpoint serialization, and final artifact writing. A secondary active end-to-end rate divided training target tokens by the active training-loop wall time, including evaluation, logging, checkpointing, and batch acquisition. Neither metric includes model construction or initial data-loader setup, and neither is an inference benchmark.

Training and evaluation peak GPU allocation were measured separately with `torch.cuda.max_memory_allocated` [14], resetting the counter at phase boundaries. The primary peak-memory analysis uses peak training allocation. Evaluation peak allocation is retained as a secondary systems measurement. These values exclude PyTorch’s unallocated reserved cache, other device processes, and host-system memory.

### 3.5.4 Routing and MTP Diagnostics

For router probabilities  $p_{ie}$  over  $E$  routed experts, mean routing entropy was

$$H_{\text{route}} = -\frac{1}{N} \sum_{i=1}^N \sum_{e=1}^E p_{ie} \log p_{ie}. \quad (7)$$

For realized top- $k$  expert-selection fractions  $f_e$ , expert-load variance was

$$V_{\text{load}} = \frac{1}{E} \sum_{e=1}^E (f_e - \bar{f})^2. \quad (8)$$

Entropy describes the dispersion of router probabilities, whereas load variance describes the balance of realized expert selections. Neither quantity alone establishes useful expert specialization.

The retained routing statistics are latest-forward snapshots. For each run, the final values were taken after test evaluation, averaged across the 12 MoE layers, and then summarized across seeds. They are not averages over the full training trajectory. The reported MTP auxiliary loss is similarly the final optimizer-step value averaged across its two horizons; it is a terminal diagnostic rather than an optimization curve.

## 3.6 Statistical Analysis

### 3.6.1 Descriptive Summaries

For each model–budget cell, metrics were summarized across the ten seeds using the arithmetic mean and a two-sided 95% Student- $t$  confidence interval,

$$\bar{y} \pm t_{0.975,9} \frac{s}{\sqrt{10}}, \quad t_{0.975,9} = 2.262157. \quad (9)$$

The seed is the experimental unit. Tokens, batches, and evaluation windows were not treated as independent replicates.

### 3.6.2 Omnibus Model Comparisons

Within each budget, a one-factor repeated-measures analysis of variance tested whether the mean outcome differed across the six models, with seed treated as the block. A Friedman rank test provided a nonparametric companion. For both statistics, 20,000 random permutations were formed by relabeling the six model outcomes within each seed while preserving the seed blocks. If  $r$  permuted statistics were at least as extreme as the observed statistic, the reported permutation value was

$$p_{\text{perm}} = \frac{r + 1}{20,000 + 1}. \quad (10)$$

These omnibus tests establish whether model choice affects the outcome before interpreting specific mechanism comparisons.

### 3.6.3 Planned Paired Contrasts

Seven comparisons were specified to match the controlled mechanism questions: MLA–Dense, MTP–Dense, MoE–Dense, MLA+MoE–Dense, MLA+MoE–MLA, MLA+MoE–MoE, and V3 Routing–MoE. For aligned seed  $i$ ,

$$d_i = Y_{i,B} - Y_{i,A}, \quad \bar{d} = \frac{1}{10} \sum_{i=1}^{10} d_i. \quad (11)$$

For loss,  $\bar{d} < 0$  favors Model B. Each contrast reports the mean paired difference, the number of seeds in which Model B performed better, a 20,000-resample paired percentile-bootstrap 95% interval [6], and the standardized paired effect

$$d_z = \frac{\bar{d}}{s_d}, \quad (12)$$

where  $s_d$  is the sample standard deviation of the ten paired differences. An exact two-sided sign-flip test enumerated all  $2^{10} = 1,024$  assignments of signs to the paired differences. Its minimum positive two-sided  $p$ -value was  $2/2^{10} = 0.001953125$  when no zero-valued assignments were counted.

### 3.6.4 Multiplicity Control

Test loss was the primary inferential endpoint. Principal significance claims use Holm’s sequential adjustment [10] across the complete family of 21 planned test-loss comparisons. This family was chosen to control the full primary mechanism question set rather than selecting a budget after inspecting the results. Budget-specific and broader metric-group adjustments were retained as supplementary sensitivity analyses. Perplexity was not included as an additional primary family because it is determined directly by test loss.

## 3.7 Hardware, Reproducibility, and Implementation Boundaries

All 180 runs were executed in full 32-bit precision on one NVIDIA GeForce RTX 4050 Laptop GPU under the recorded Windows platform, Python 3.11.9, PyTorch 2.6.0+cu124, CUDA 12.4, and zero data-loader workers. Deterministic algorithms were enabled, TF32 and cuDNN benchmarking were disabled, and CUBLAS\_WORKSPACE\_CONFIG was :4096:8. Every run summary records the clean training commit `b40a593fc1125c109c96fe18d07e594bc79c9ec9`, source-tree hash, resolved configurations, dependency-lock hashes, runtime fingerprint, data and tokenizer hashes, evaluation-window hashes, and sampler state. The canonical evidence was audited from current repository state `2c01074fdfe81d30503f6bef00816f4146b5e04a`. The repository is available at <https://github.com/mmiovski/deepseek-reimplementation>.

A post-run audit verified the balanced 180-run matrix, canonical configuration closure, artifact provenance, metric and figure hashes, and tracked analysis outputs. The completed matrix passes the 291-test suite, and the extraction, descriptive, global-test, paired-contrast, budget-trend, mechanism-profile, and evidence-lineage audits report no missing primary metrics or failed cells. These controls support protocol-level reproducibility but seeded GPU execution is not guaranteed to be bitwise identical across hardware, operating systems, CUDA versions, or library versions.

The implemented models are local DeepSeek-inspired analogues rather than full reproductions of DeepSeek-V2, DeepSeek-V3, or DeepSeek-R1. The study does not include distributed expert parallelism,

expert-capacity enforcement, token dropping, custom sparse kernels, production cache compression, FlashMLA, FP8 training, frontier-scale sequential MTP modules, speculative decoding, instruction tuning, reinforcement learning, or reasoning evaluation. These boundaries keep the experiment focused on mechanism behavior under constrained local compute; their consequences for external validity are addressed in the limitations section.

## 4. Results and Discussion

This section answers the study questions through the prespecified paired comparisons. Test loss is the primary inferential outcome. Unless stated otherwise, reported significance decisions use the Holm adjustment across all 21 planned test-loss contrasts. For each contrast, the reported difference is Model B minus Model A so a negative value favors Model B. Mechanistic explanations are presented as interpretations of the observed patterns rather than causal proof.

### 4.1 Overall Quality and Token-Budget Scaling

All six models improved as the training-token budget increased. Figure 1 shows a consistent downward shift in mean test loss from 10M to 25M and again from 25M to 50M tokens. From 10M to 50M, mean test loss decreased by 0.592–0.634 across the six models, and every model improved for all ten aligned seeds. The exact sign-flip  $p$ -value for each 10M-to-50M paired change was  $p = 0.001953$ . Because every budget used the same fixed held-out windows, these changes are directly comparable rather than being partly attributable to different evaluation prefixes.

MoE had the lowest mean test loss at every budget: 4.94357 at 10M, 4.53808 at 25M, and 4.35185 at 50M. V3 Routing remained close to MoE, with means of 4.95151, 4.53894, and 4.35484. Dense and MTP were nearly identical at 10M; MTP moved below Dense at 25M and 50M. The MLA-based models remained weaker than the corresponding dense-attention alternatives.

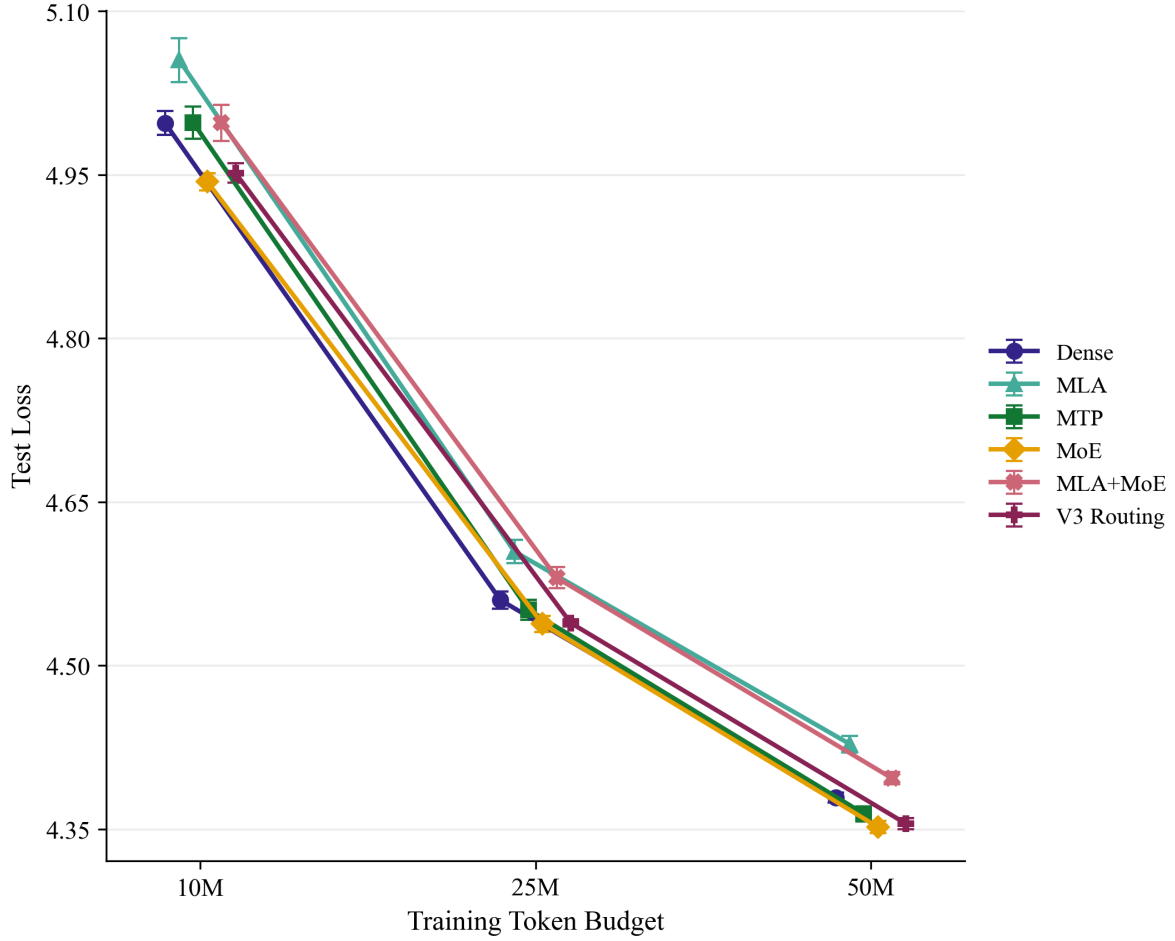
The global tests confirmed a model effect on test loss at every budget. Both the repeated-measures ANOVA and the rank-based Friedman test were significant under within-seed permutation at 10M, 25M, and 50M tokens (Table 3). Their agreement shows that the conclusion was not limited to the mean-based omnibus statistic.

**Table 3:** Global model-effect tests for test loss at each token budget. Both tests preserve the aligned-seed block structure. Permutation  $p$ -values use 20,000 within-seed model-label permutations.

| Budget | RM-ANOVA $F(5, 45)$ | Perm. $p$ | Friedman $Q(5)$ | Perm. $p$ |
|--------|---------------------|-----------|-----------------|-----------|
| 10M    | 38.984              | 0.000050  | 41.657          | 0.000050  |
| 25M    | 60.236              | 0.000050  | 43.371          | 0.000050  |
| 50M    | 170.356             | 0.000050  | 45.314          | 0.000050  |

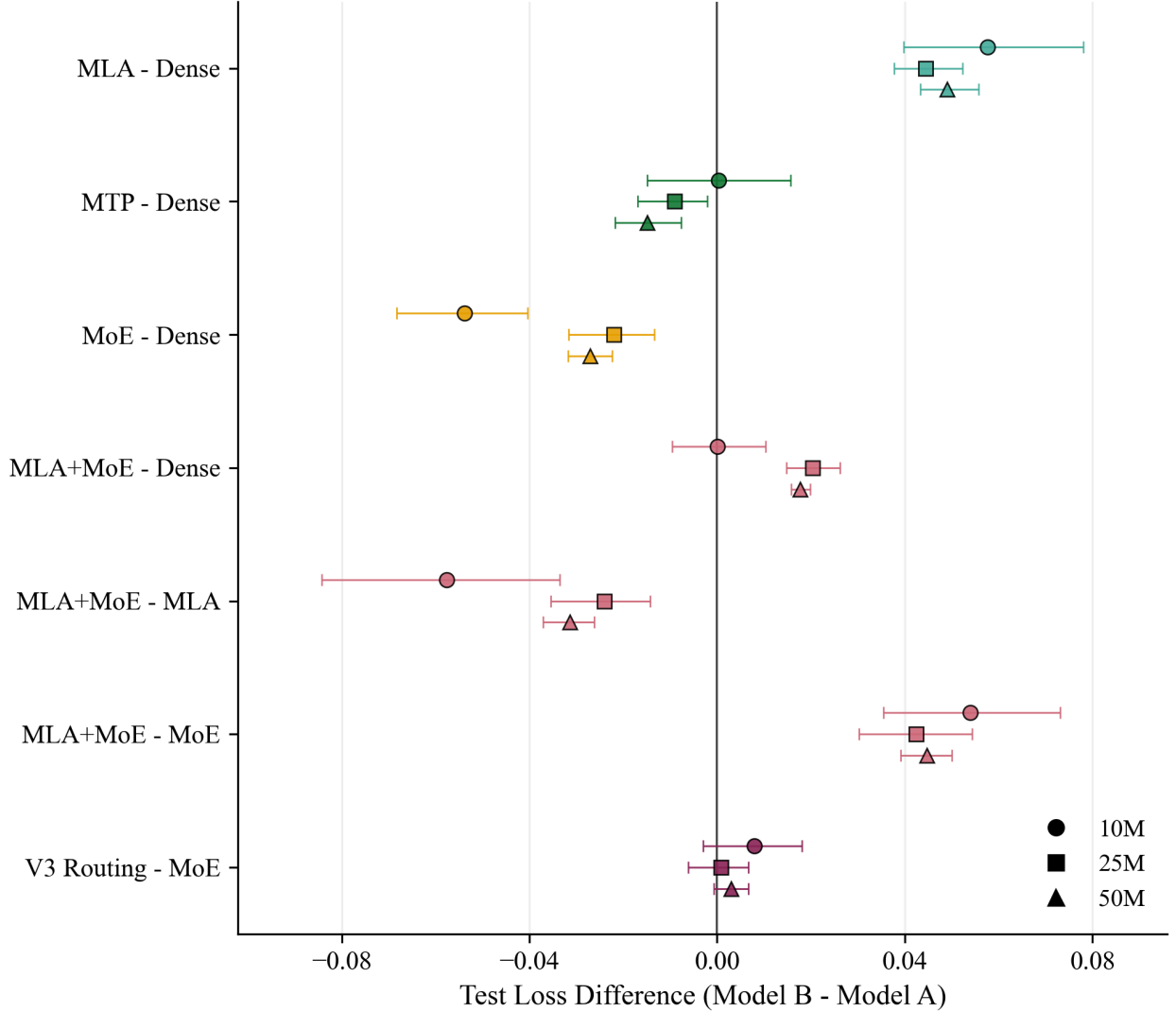
The repeated-measures ANOVA tests mean differences among the six models; the Friedman test provides a rank-based companion. The consistent omnibus results justify interpretation of the prespecified paired contrasts rather than post hoc selection of model pairs.

The planned contrasts in Figure 2 identify where those global differences arose. MoE improved on Dense at all three budgets. The mean differences were  $-0.054$ ,  $-0.022$ , and  $-0.027$ , every one of the 30 aligned seed pairs favored MoE, and all three comparisons remained significant under the 21-comparison



**Figure 1:** Mean test next-token cross-entropy across nominal token budgets; lower is better. Error bars are 95% Student- $t$  intervals across ten aligned seeds. Small horizontal offsets separate model markers at the same nominal budget and have no quantitative meaning. Every budget uses the same 400 fixed, non-overlapping, corpus-spanning test windows.

Holm adjustment ( $p = 0.041016$ ). MLA was worse than Dense at all budgets, by 0.058, 0.044, and 0.049, with all aligned pairs favoring Dense and all adjusted values equal to 0.041016. MTP did not differ from Dense at 10M and its 25M advantage did not survive the primary adjustment, but at 50M it reduced mean test loss by 0.015, won nine of ten seed pairs, and remained significant after adjustment ( $p = 0.041016$ ). V3 Routing did not establish an advantage over ordinary MoE at any budget.



**Figure 2:** Prespecified paired test-loss contrasts across token budgets. Differences are Model B minus Model A so negative values favor Model B. Intervals are paired percentile-bootstrap 95% confidence intervals from 20,000 resamples. Budget markers are separated slightly for readability only. Statistical conclusions use the prespecified 21-comparison Holm family and are not determined solely by whether an individual bootstrap interval crosses zero.

## 4.2 MLA and Sparse-Capacity Tradeoffs

The MLA comparison asks whether the local latent-attention analogue improved training memory or throughput relative to Dense. It did not. MLA increased test loss by 0.058, 0.044, and 0.049 at 10M, 25M, and 50M, respectively. The contrasts were significant at every budget. MLA was also 12.6–12.8%

slower in synchronized training-step throughput and used 15.0% more peak training allocation. At 50M, its mean training and evaluation peaks were 3.31 and 2.30 GiB, compared with 2.88 and 2.01 GiB for Dense.

This should be interpreted against MLA’s original motivation. In DeepSeek-V2, MLA was introduced primarily to compress the key–value cache during autoregressive inference, thereby reducing cache storage and memory-bandwidth pressure [3]. The present experiment measured full-sequence training, not cached autoregressive decoding. Its local MLA path projected hidden states into a latent representation and then reconstructed per-head keys and values on every forward pass; it did not use a production KV cache, FlashMLA, fused kernels, or the deployment stack that creates the original inference advantage. The MLA model also contained 137.460 million parameters compared with 120.945 million for Dense. Extra projection work and stored parameters therefore remained visible in training memory and step time while the intended decode-time cache benefit was not exercised.

The supported conclusion is correspondingly narrow. The MLA analogue did not improve predictive quality, training-step throughput, active end-to-end throughput, peak training allocation, or evaluation allocation under the local protocol. It does not refute MLA’s production inference objective. The design also does not isolate whether its quality gap arose from the latent attention geometry, increased parameterization, optimization at the selected budgets, or an interaction among these factors.

MoE produced a different tradeoff. The model stored 220.146 million total parameters but activated an estimated 78.589 million per token, or 35.7% of stored capacity. At 50M tokens, this corresponded to 0.227 tokens per total parameter but 0.636 tokens per activated parameter, compared with 0.413 under both definitions for Dense. The architecture consequently achieved the intended separation between total capacity and per-token activated capacity that motivates sparse conditional computation [2, 7].

The paired evidence establishes a predictive-quality advantage for MoE over Dense at every budget. That advantage did not become a systems advantage. MoE was 47.9–48.0% slower in synchronized training-step throughput and used 50.7% more peak training allocation. At 50M, mean active end-to-end throughput was 2,162.0 tokens/s for MoE versus 4,144.7 for Dense, while mean evaluation allocation was 3.54 versus 2.01 GiB. Sparse routing avoids applying every routed expert to every token, but all expert parameters remain resident and trainable. In this single-GPU implementation, routing also required top- $k$  selection, indexing, repeated small expert operations, weighted aggregation, and auxiliary-loss computation without custom sparse kernels or expert parallelism.

The answer to the sparse-routing question is therefore mixed in a precise way. MoE achieved lower test loss and greater token exposure per activated parameter while storing a larger, more weakly exposed parameter bank. It did not deliver faster training or lower allocated memory. Whether this constitutes greater efficiency depends on whether predictive quality, activated-path accounting, storage, memory, or wall-clock step rate is the operative constraint.

### 4.3 Interaction Between MLA and MoE

The combined MLA+MoE model completed all 30 planned runs, demonstrating technical composability. Its paired results show that the MoE feed-forward path improved the MLA model at every budget. Relative to MLA, MLA+MoE reduced test loss by 0.058, 0.024, and 0.031; nine, nine, and ten of the aligned seed pairs favored the combined model, and all three comparisons remained significant after the primary Holm adjustment.

The combination nevertheless remained worse than ordinary MoE by 0.054, 0.042, and 0.045 across the three budgets, with every aligned seed favoring MoE and all adjusted  $p$ -values equal to 0.041016.

Relative to Dense, MLA+MoE was indistinguishable at 10M but worse by 0.020 and 0.018 at 25M and 50M. Thus, adding MoE recovered some of the MLA quality deficit, but adding MLA to MoE degraded quality under the selected configuration.

The systems profile also retained the costs of both mechanisms. MLA+MoE stored 236.662 million parameters and activated 95.104 million per token. It was approximately 51.2% slower than Dense and 6.4% slower than MoE in synchronized training-step throughput, while peak training allocation was 61.4% above Dense and 7.1% above MoE. This is evidence against a compounded quality–systems benefit in the tested configuration, not proof of an inherently antagonistic interaction. The models were not constructed as a fully crossed, matched-capacity factorial design, so parameter count and mechanism cannot be separated perfectly.

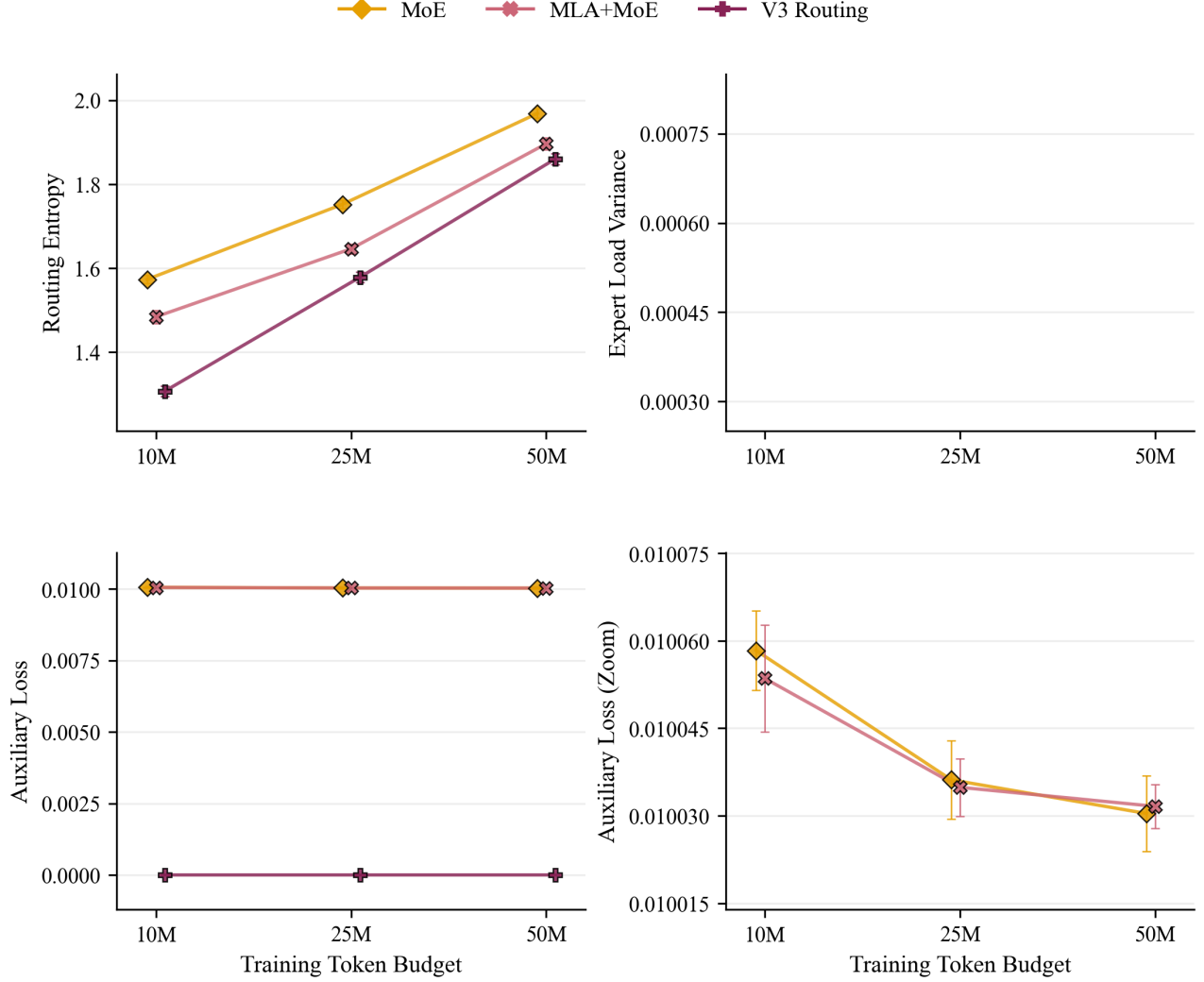
#### 4.4 V3-Style Routing Behavior

V3 Routing held the MoE expert geometry fixed and replaced the differentiable load-balancing loss with a non-gradient expert-selection bias. Its test-loss differences relative to ordinary MoE were 0.008, 0.001, and 0.003 from 10M through 50M tokens. None survived the primary adjustment, and their bootstrap intervals included zero. The routing change therefore did not improve next-token quality.

It also did not establish a training systems advantage. Relative to MoE, V3 Routing was 1.5% slower at 10M, 0.04% faster at 25M, and 0.09% faster at 50M; all adjusted throughput values were 1.000000. Peak training allocation differed by less than 0.02% at every budget, again without a statistically significant difference. Removing the auxiliary-loss term was consequently too small a change to alter the dominant expert-storage and dispatch costs in a reliable way.

Figure 3 shows the terminal routing distributions. Routing entropy increased with budget for all three sparse models. At 50M, mean entropy was 1.96881 for MoE, 1.89646 for MLA+MoE, and 1.85985 for V3 Routing. Mean expert-load variance was low in absolute magnitude and decreased from 10M to 50M for each sparse model. V3 Routing moved from 0.000076 to 0.000054, while ordinary MoE moved from 0.000062 to 0.000048. The V3 controller therefore ended with descriptively lower entropy and slightly higher load variance than MoE, but these final-batch diagnostics are not primary inferential endpoints and do not prove collapse, specialization, or trajectory stability.

This outcome is consistent with the difference between the two controllers. The auxiliary-loss model receives direct gradient pressure linking routing probabilities and expert load, whereas V3 Routing adjusts only a selection bias after each training forward. DeepSeek-V3 introduced auxiliary-loss-free balancing to avoid the possibility that a balancing objective interferes with the language-model objective at large scale [4]. In the local experiment, removing that objective did not improve quality or systems behavior, and the retained terminal snapshots do not show a balance advantage over ordinary MoE.

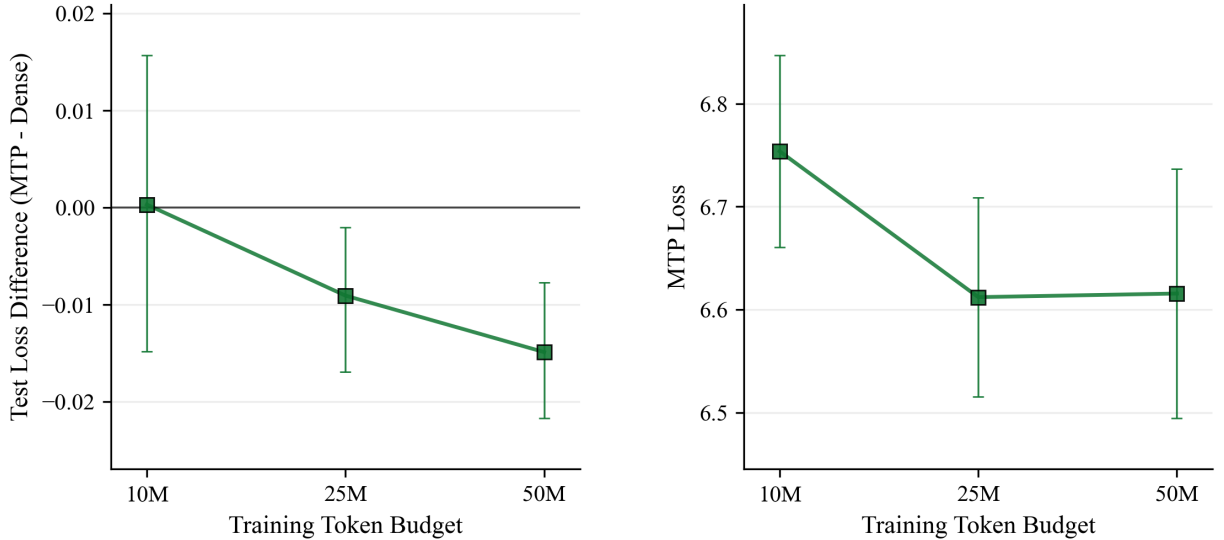


**Figure 3:** Terminal routing diagnostics for the three sparse models. Values come from the latest forward pass of the final test evaluation, averaged across 12 MoE layers and then across ten seeds; error bars are 95% Student- $t$  intervals. Small model offsets are visual only. The expert-load-variance panel is zoomed to distinguish low-magnitude values; the underlying values are unchanged. V3 auxiliary loss is zero by design.

## 4.5 Multi-Token Prediction

MTP tested whether two auxiliary future-token heads improved optimization or sample efficiency. Its mean test-loss differences relative to Dense were 0.000 at 10M,  $-0.009$  at 25M, and  $-0.015$  at 50M. The 10M models were effectively indistinguishable. At 25M, MTP won eight of ten seed pairs and its bootstrap interval excluded zero, but the exact  $p = 0.044922$  did not survive the complete primary Holm family ( $p = 0.269531$ ). At 50M, MTP won nine of ten pairs; the bootstrap interval was  $[-0.022, -0.008]$ , the exact  $p$ -value was 0.005859, and the primary adjusted value was 0.041016. The evidence therefore establishes a modest next-token advantage at 50M, not a budget-general advantage.

The terminal auxiliary-loss diagnostic does not show a monotonic improvement across all budgets. Mean final-step MTP loss was 6.75370 at 10M, 6.61215 at 25M, and 6.61568 at 50M. The 50M-minus-10M mean change was  $-0.138$ , but its paired 95% interval included zero and the exact sign-flip value was 0.093750. This terminal value is sensitive to the final sampled optimizer batch and should not be treated as an MTP learning curve. The next-token result at 50M is therefore the stronger evidence; the retained auxiliary snapshot only confirms that the two auxiliary heads produced finite, measurable losses.

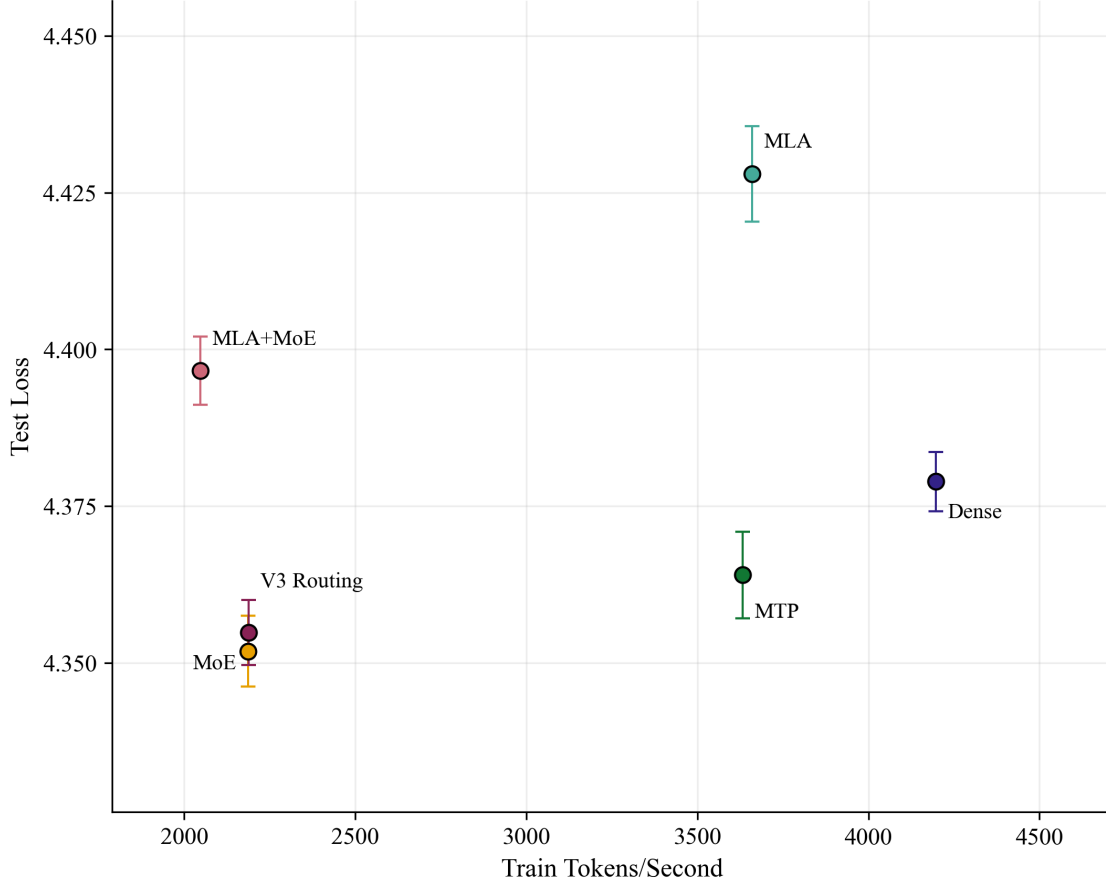


**Figure 4:** MTP behavior across token budgets. Left: paired MTP-minus-Dense test-loss differences with 20,000-resample percentile-bootstrap 95% intervals; negative favors MTP. Right: final optimizer-step MTP auxiliary loss with 95% Student- $t$  intervals across ten seeds.

The local cost remained clear. The two full-vocabulary auxiliary heads added 15.360 million parameters relative to Dense, increased peak training allocation by 12.9%, and reduced synchronized training-step throughput by 13.5–13.7%. At 50M, mean active end-to-end throughput was 3,590.3 tokens/s for MTP versus 4,144.7 for Dense. The MTP research question is consequently budget-dependent: the auxiliary objective imposed measurable training cost and yielded a next-token improvement at 50M, but the experiment does not establish the same advantage at shorter budgets and provides no evidence about speculative decoding or reasoning.

## 4.6 Cross-Mechanism Quality, Systems Cost, and Exposure Synthesis

The 50M endpoint summarizes the tradeoff among the six models. Figure 5 places MoE at the lowest mean test loss and Dense at the highest synchronized training-step throughput. MTP occupied an intermediate position, establishing lower test loss than Dense but at a 13.5% step-throughput cost. MLA was both slower and less accurate than Dense. MLA+MoE improved on MLA in quality but combined relatively high loss with the lowest throughput. V3 Routing remained statistically indistinguishable from MoE in both quality and throughput.



**Figure 5:** Quality–throughput tradeoff at the 50M-token budget. The horizontal axis is mean synchronized training-step throughput; the vertical axis is mean test loss, with 95% Student- $t$  intervals across ten seeds. Higher throughput and lower loss are favorable, but the plot does not define a single scalar ranking. Horizontal uncertainty is not shown.

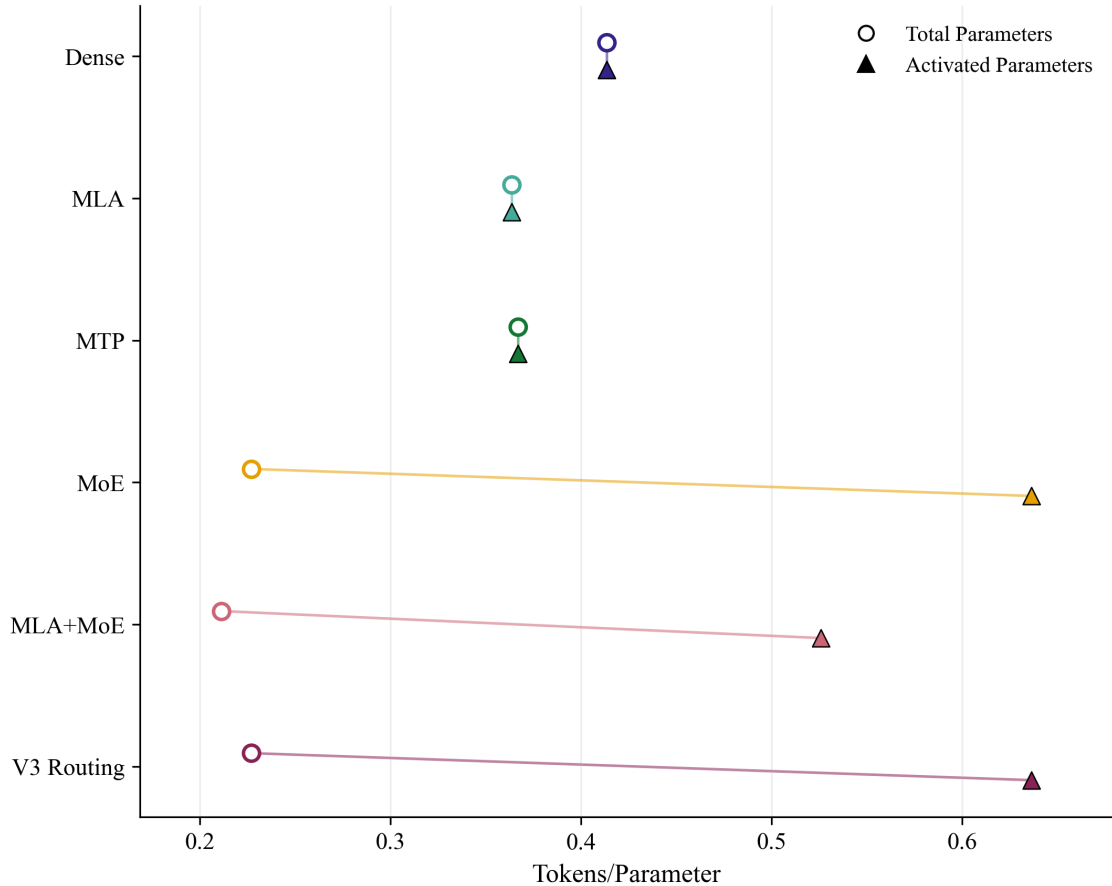
Parameter exposure explains why total model size alone is insufficient for this comparison. Figure 6 separates tokens per total parameter from tokens per activated parameter. The two measures coincide for Dense, MLA, and MTP because every parameter participates in every token path. They diverge sharply for MoE and V3 Routing. Each stored 220.146 million parameters but only an estimated 78.589 million were activated per token. MLA+MoE showed the same pattern at a higher activated count.

At 50M, MoE had fewer tokens per total parameter than Dense (0.227 versus 0.413), making it more data-limited relative to stored capacity. It simultaneously had more tokens per activated parameter (0.636 versus 0.413) because each token used only part of the expert pool. Sparse models can therefore appear heavily overparameterized under total capacity while receiving greater exposure per active path.

**Table 4:** Quality, training systems cost, and parameter exposure at the 50M-token budget. Values are means across ten aligned seeds; test-loss intervals are 95% Student- $t$  intervals. Throughput uses synchronized optimizer-step time, and peak memory is maximum training-phase CUDA memory allocated by PyTorch.

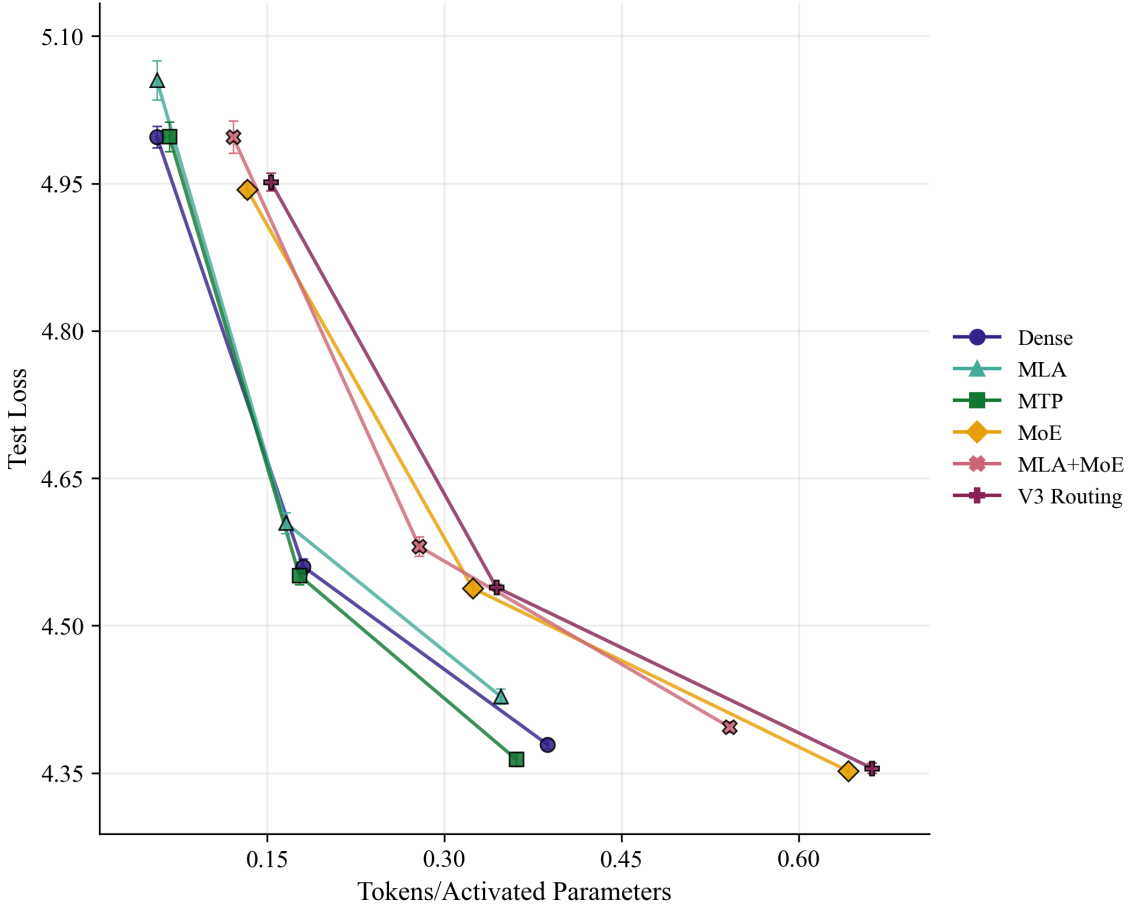
| Model      | Test loss [95% CI]         | Test PPL | Step tokens/s | Train GiB | Tokens/total | Tokens/activated |
|------------|----------------------------|----------|---------------|-----------|--------------|------------------|
| Dense      | 4.37893 [4.37419, 4.38366] | 79.75383 | 4196.9        | 2.88      | 0.413        | 0.413            |
| MLA        | 4.42797 [4.42035, 4.43559] | 83.76537 | 3659.7        | 3.31      | 0.364        | 0.364            |
| MTP        | 4.36401 [4.35711, 4.37090] | 78.57476 | 3632.2        | 3.25      | 0.367        | 0.367            |
| MoE        | 4.35185 [4.34620, 4.35749] | 77.62375 | 2186.8        | 4.34      | 0.227        | 0.636            |
| MLA+MoE    | 4.39658 [4.39114, 4.40203] | 81.17524 | 2047.5        | 4.64      | 0.211        | 0.526            |
| V3 Routing | 4.35484 [4.34966, 4.36002] | 77.85631 | 2188.7        | 4.34      | 0.227        | 0.636            |

All parameters were trainable, so tokens per trainable parameter equal tokens per total parameter. Exposure ratios use the actual 50M-run token count of 50,000,896. Activated parameters are an analytical per-token architecture estimate, not measured FLOPs. Mean active end-to-end rates at 50M were 4144.7, 3612.7, 3590.3, 2162.0, 2023.6, and 2164.3 tokens/s in table order; mean evaluation peaks were 2.01, 2.30, 2.24, 3.54, 3.81, and 3.54 GiB.



**Figure 6:** Total-capacity and activated-compute exposure at 50M tokens. Values are deterministic for a given architecture and processed-token count so no uncertainty intervals are shown. Marker offsets distinguish the two exposure definitions within each model and do not represent different budgets. Total and trainable exposure are identical because all parameters were trainable.

Neither ratio alone determines quality. Figure 7 shows that models with similar activated exposure still differed in loss and models with different exposure could achieve similar loss.



**Figure 7:** Mean test loss versus tokens per activated parameter across models and budgets. Vertical bars are 95% Student- $t$  intervals across ten seeds; small horizontal offsets are display-only.

Taken together, the results answer the central research question in mechanism-specific terms rather than through a universal ranking. MoE provided the best predictive quality at every budget and a confirmed advantage over Dense, but its local training path was substantially slower and more memory-intensive. Dense provided the best training systems profile. MTP produced a smaller, budget-specific quality gain at 50M with moderate systems cost. MLA did not realize its intended inference-oriented benefit in the measured training setting. Adding MoE improved MLA, but the combination remained worse than ordinary MoE and expensive to execute. V3-style bias routing did not improve on the auxiliary-loss MoE baseline.

The most universal effect remained additional training data: every model improved in every aligned seed, with mean test loss decreasing monotonically across the three budgets. The study did not include a same-architecture capacity sweep dense enough to identify an interpolation threshold or support classical double-descent claims. The findings should therefore be read as controlled local evidence about several distinct efficiency dimensions, not as a scalar leaderboard, a general scaling law, or a reproduction of production DeepSeek systems.

## 5. Limitations and Threats to Validity

The study was designed to isolate several DeepSeek-inspired mechanisms under a controlled local protocol, not to reproduce the production systems from which those mechanisms were drawn. The balanced design, aligned seeds, adjacent-pair positional encoding, fixed held-out sample, and self-authenticating artifacts strengthen the internal comparisons but they do not remove the limits imposed by model scale, implementation choices, measurement design, and hardware. Conclusions should be read as evidence about the six implemented systems within this regime.

### 5.1 Scale and Implementation Fidelity

The models were trained at approximately 121–237 million total parameters, with a 256-token context and at most 50 million training tokens. These settings create a deliberately data-limited and overparameterized local regime but they are far from the scale, context length, data diversity, and optimization infrastructure used by production language models. A mechanism that is costly at this scale may become useful when communication, memory bandwidth, cache size, or expert specialization becomes the dominant constraint. Conversely, a local quality gain need not persist when the same mechanism is distributed across many devices or trained for substantially more tokens.

The implemented MLA variant was a local latent-projection analogue rather than the complete DeepSeek-V2 attention stack. It did not evaluate autoregressive key–value cache size, decode latency, FlashMLA, fused kernels, or production cache management. Negative training-memory and throughput results therefore apply to the implemented full-sequence training path, not to MLA’s intended inference-time cache advantage. The sparse models likewise omitted expert parallelism, distributed all-to-all communication, custom sparse kernels, capacity enforcement, and token dropping. Their measured overhead reflects a single-GPU implementation in which all experts remained resident and routed computation was executed through general PyTorch operations. It should not be interpreted as a benchmark of a production MoE systems stack.

The MTP implementation used two separate full-vocabulary auxiliary heads at offsets two and three. It did not reproduce the sequential MTP modules used by DeepSeek-V3, generation-time token verification, speculative decoding, instruction tuning, reinforcement learning, or reasoning evaluation. The experiment can determine whether this local auxiliary objective changed next-token learning within the tested budgets, but it cannot determine whether production MTP would improve decoding efficiency or downstream reasoning behavior.

### 5.2 Measurement and Evaluation Design

Training examples were random windows sampled with replacement from one tokenized corpus. Windows could overlap or repeat, so processed tokens were dependent language-model targets rather than independent observations. The statistical analysis treated the ten aligned seeds as the experimental units, but the effective diversity of the training signal remained constrained by a single corpus, tokenizer, and document sample.

Validation and test each used 400 deterministic, non-overlapping, corpus-spanning windows, and the identical window set was used at 10M, 25M, and 50M. This supports clean within-budget model comparisons and cross-budget descriptive changes. The evaluation is still a fixed sample of 102,400 target tokens per split rather than an exhaustive pass over the held-out corpora, and repeated use of

the same validation sample during training can make it less independent than the final test sample.

The systems measurements have narrow meanings. Primary throughput counts training target tokens per synchronized optimizer-step second, including data transfer, forward and backward computation, gradient clipping, and the optimizer update while excluding evaluation, logging callbacks, checkpoint serialization, and final artifact writing. Active end-to-end throughput includes those recurring loop costs but still excludes model construction and initial data-loader setup. Neither quantity is an inference-latency, energy, kernel-only, or distributed-throughput benchmark. Training and evaluation peak allocation were recorded separately, but both use maximum CUDA tensor memory allocated by PyTorch rather than reserved device memory, host memory, or total process occupancy. Activated parameter count is an analytical architecture-level estimate rather than measured floating-point work, communication, or kernel efficiency.

The retained mechanism diagnostics are terminal summaries rather than training trajectories. Routing entropy and expert-load variance were measured from the final test evaluation and averaged across layers, while MTP loss was taken from the final optimizer step. They can describe the state reached by each implementation, but they cannot establish when routing stabilized, whether experts developed persistent specialization, or how the auxiliary objective evolved. The final MTP value is especially sensitive to the last randomly sampled training batch. Durable logs retain more trajectory information, but the canonical report analysis intentionally uses the audited summary-level diagnostics.

Finally, ten seeds provide a substantially stronger basis than a single-run comparison but they still limit inferential resolution. The exact two-sided sign-flip test has discrete support, and confidence intervals can remain wide for small effects. The Holm adjustment controls the complete primary family but also reduces power. A nonsignificant contrast should therefore be interpreted as a failure to establish an advantage rather than proof that two models are equivalent. Conversely, a statistically consistent systems difference may be practically negligible, as in the sub-0.02% peak-memory differences between the two MoE routing controllers.

### 5.3 Generalization and Untested Claims

All final runs used full 32-bit precision on one NVIDIA RTX 4050 Laptop GPU under one recorded Windows, CUDA, and PyTorch environment. The throughput and memory results are consequently hardware-specific and implementation-specific. The study also used one FineWeb-Edu sample, one 10,000-token BPE tokenizer, one context length, one optimizer configuration, and three short-to-medium token budgets. Generalization across corpora, tokenizer sizes, sequence lengths, precision formats, accelerators, or larger training budgets remains untested.

The experiment contains two broad total-parameter scales but capacity is confounded with mechanism: the larger models are the sparse variants rather than larger dense versions of the same architecture. The matrix therefore cannot identify an interpolation threshold, estimate a capacity scaling curve, or establish classical double descent. It also cannot determine whether MoE’s quality advantage arose from sparse conditional computation specifically, from larger stored capacity, or from their combination.

The paired design supports controlled empirical comparisons but it does not by itself prove the causal mechanism behind every observed difference. MLA changes both attention structure and parameter count, while MoE changes feed-forward structure, total capacity, routing, and execution pattern. Explanations in Section 4 are consistent with the implementation and measurements, but additional ablations would be needed to isolate projection dimensions, expert counts, top- $k$ , auxiliary-loss strength, bias-update rate, MTP weight, or parameter-matched controls.

Atomic checkpoints, exact sampler-state restoration, run-identity validation, and deterministic settings support interruption-level resumption on the retained setup. They do not guarantee bitwise identity across different machines, drivers, accelerators, operating systems, CUDA kernels, or library versions. More broadly, the study does not evaluate instruction following, downstream tasks, calibration, generation quality, inference latency, speculative decoding, or reasoning. Its conclusions are limited to next-token prediction, measured local training cost, parameter exposure, and terminal mechanism diagnostics under the study protocol.

## 6. Conclusion

### 6.1 Main Findings

This study evaluated six DeepSeek-inspired decoder variants across three token budgets and ten aligned seeds, producing a balanced set of 180 completed runs. The central result is not that one efficiency mechanism dominated every metric. Rather, the mechanisms redistributed stored capacity, activated capacity, predictive quality, optimization signal, memory demand, and execution overhead in different ways. Their value depended on the bottleneck being measured and on whether the local implementation exercised the systems advantage for which the mechanism was originally designed.

Additional training tokens produced the most universal improvement. All six model families reduced test loss from 10M to 50M for every aligned seed, with identical held-out samples across budgets. Architecture still affected the endpoint. MoE had the lowest mean test loss at every budget and achieved an advantage over Dense in all three paired comparisons. MTP also established a smaller advantage over Dense at 50M, although not at 10M or 25M under the complete primary family.

Those quality gains had distinct systems costs. Dense achieved the highest synchronized training-step throughput, the highest active end-to-end rate, and the lowest training and evaluation peak GPU allocation. MoE was approximately 48% slower in training-step throughput and used 50.7% more peak training allocation despite activating only 35.7% of its stored parameters per token. MTP was 13.5–13.7% slower and used 12.9% more peak training allocation than Dense. The results therefore do not support a scalar overall ranking. MoE led predictive quality and tokens per activated parameter; Dense led local training speed and memory; MTP provided a smaller, budget-specific quality improvement with a moderate systems penalty.

The MLA analogue did not improve any measured training-efficiency outcome. Relative to Dense, it produced higher test loss, lower training-step and end-to-end throughput, and greater training and evaluation allocation at all three budgets. This finding applies to the implemented full-sequence training path. It does not test or reject MLA’s intended production benefit of reducing the autoregressive key–value cache because cached decoding, FlashMLA, and fused inference kernels were outside the experiment.

Combining MLA with MoE produced an informative asymmetric result. The MoE feed-forward path improved MLA at every budget, but the combined model remained worse than ordinary MoE and was the slowest, most memory-intensive variant. V3-style expert-bias routing likewise failed to improve on its matched MoE baseline. It produced no reliable test-loss, throughput, or memory advantage, and its final expert-load variance remained descriptively above ordinary MoE. These terminal diagnostics do not prove routing collapse or failed specialization; they show only that the implemented bias controller did not improve the measured endpoints within the tested regime.

MTP’s 50M quality result should be separated from its auxiliary diagnostic. The final-step auxiliary

loss was lower at 25M and 50M than at 10M on average, but the 10M-to-50M paired change was not significant and the series was not monotonic. The terminal auxiliary snapshot therefore does not establish convergence behavior. The primary test result nevertheless shows that the local objective produced a modest next-token benefit at the longest tested budget, at measurable parameter, memory, and throughput cost. The experiment provides no evidence about speculative decoding or reasoning, which were not evaluated.

The parameter-exposure analysis clarifies why these results cannot be reduced to model size. Tokens per total parameter describe how strongly stored capacity is exposed to data, whereas tokens per activated parameter approximate exposure of the path used for each token. These quantities coincide for Dense, MLA, and MTP but diverge sharply for sparse models. MoE achieved lower loss with fewer tokens per stored parameter and more tokens per activated parameter than Dense, but the design cannot attribute that result uniquely to sparse activation rather than larger total capacity.

Taken together, the evidence answers the central research question in mechanism-specific terms. Sparse MoE capacity produced a reliable quality benefit but required substantially greater local time and memory. Dense execution remained the most favorable local training systems choice. MLA’s intended inference advantage was not exercised, the combined MLA+MoE model did not match ordinary MoE, V3-style routing did not improve its matched baseline, and MTP yielded a smaller benefit only at the largest tested budget. Architectural motivation should therefore not be treated as a single empirical notion of efficiency: predictive quality, training-step throughput, end-to-end rate, peak allocation, stored parameters, activated parameters, routing behavior, and auxiliary-objective behavior must be measured separately.

## 6.2 Implications and Future Work

The practical implication is that mechanism selection should follow the actual computational constraint. For local full-sequence training, Dense is attractive when throughput and memory are primary. Sparse routing becomes attractive when improved predictive quality or greater stored capacity per active token justifies the systems cost, and it may become more favorable when the execution environment can exploit conditional computation through expert parallelism, optimized dispatch, and fused kernels. MLA should be judged primarily with long-context autoregressive benchmarks that measure key–value cache size, decode latency, and memory bandwidth. MTP should be evaluated over longer training horizons and with generation-time objectives before conclusions are drawn about decoding efficiency or reasoning.

A first methodological extension is to separate mechanism effects from capacity and data exposure more cleanly. Parameter-matched controls and a same-architecture capacity sweep would help determine how much of MoE’s quality advantage is attributable to larger stored capacity, sparse activation, or their interaction. Larger token budgets would test whether routed experts receive enough assignments to specialize and whether MTP’s 50M advantage strengthens, saturates, or disappears. Repeating the study across corpora, tokenizer sizes, context lengths, and precision formats would show which findings are specific to the current FineWeb-Edu sample and RTX 4050 environment.

A second direction is systems-oriented evaluation. MLA should be benchmarked with an actual autoregressive cache and optimized latent-attention kernels. MoE should be tested with expert parallelism, efficient all-to-all communication, capacity management, token dropping where appropriate, and specialized sparse kernels. Those experiments would distinguish architectural conditional-computation benefits from the overhead of the current eager single-GPU implementation. Inference latency, total

process memory, energy use, and throughput under mixed precision would provide a more complete efficiency assessment than the current training measurements alone.

Finally, richer diagnostics would strengthen the mechanism-level explanations. Checkpointed routing trajectories, per-expert token counts, specialization probes, and gradient measurements could show how expert utilization develops rather than only recording the final evaluation state. MTP learning curves, per-horizon trajectories, sequential-module implementations, and auxiliary-weight ablations could determine why the next-token advantage appeared only at 50M. Reasoning-oriented supervised fine-tuning or reinforcement learning should remain a later extension, undertaken only after the base architectural effects have been validated under broader scales and deployment-relevant systems paths.

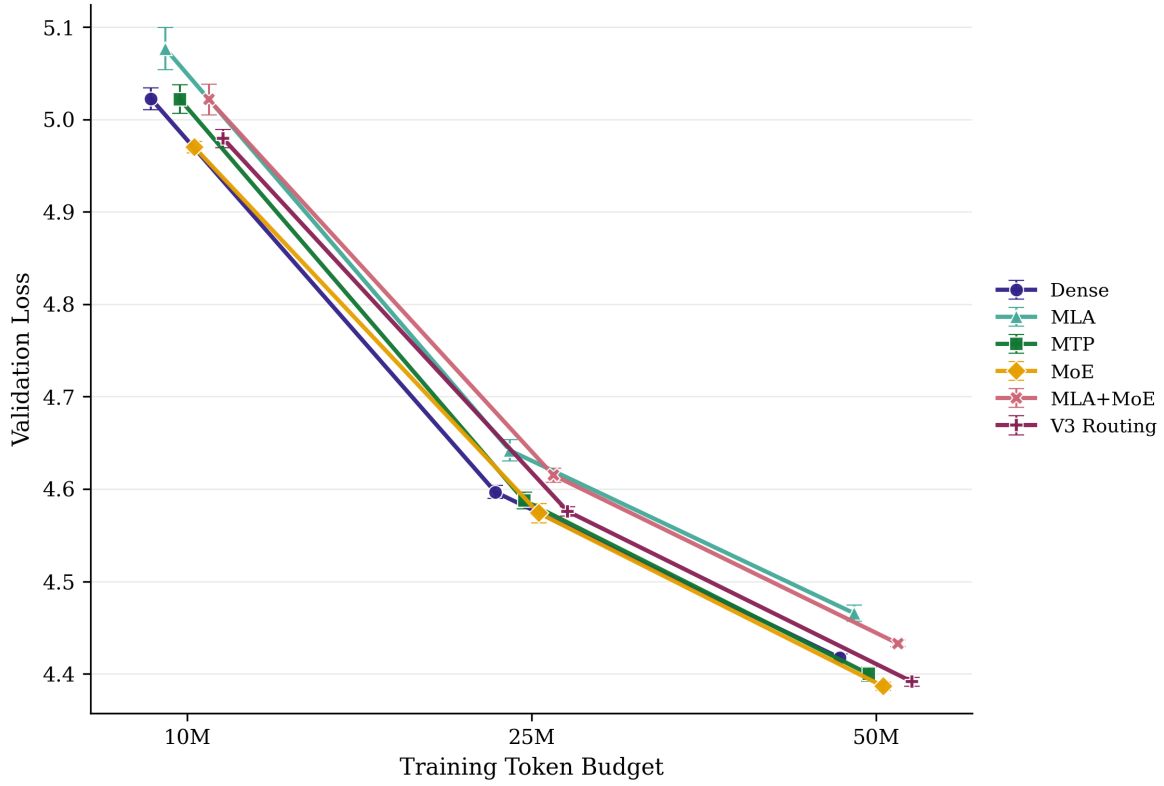
## **A. Full Descriptive Results**

This appendix preserves the exact descriptive record needed to evaluate the main quality, systems, and parameter-exposure findings. It includes all 18 model–budget cells.

**Table A.1:** Full validation and test quality descriptives. Loss entries are means with 95% Student- $t$  intervals across ten aligned seeds; perplexity entries are means. Lower values are better.

| Budget | Model      | Validation loss [95% CI]   | Test loss [95% CI]         | Validation PPL | Test PPL  |
|--------|------------|----------------------------|----------------------------|----------------|-----------|
| 10M    | Dense      | 5.02272 [5.01087, 5.03457] | 4.99745 [4.98658, 5.00832] | 151.84244      | 148.05060 |
| 10M    | MLA        | 5.07678 [5.05403, 5.09954] | 5.05513 [5.03497, 5.07529] | 160.33052      | 156.88159 |
| 10M    | MTP        | 5.02207 [5.00665, 5.03750] | 4.99774 [4.98292, 5.01256] | 151.75724      | 148.10638 |
| 10M    | MoE        | 4.97007 [4.96366, 4.97649] | 4.94357 [4.93553, 4.95160] | 144.04284      | 140.27746 |
| 10M    | MLA+MoE    | 5.02164 [5.00487, 5.03842] | 4.99751 [4.98113, 5.01390] | 151.69747      | 148.07947 |
| 10M    | V3 Routing | 4.97939 [4.96936, 4.98941] | 4.95151 [4.94252, 4.96051] | 145.39787      | 141.39857 |
| 25M    | Dense      | 4.59682 [4.58969, 4.60394] | 4.56003 [4.55227, 4.56778] | 99.17260       | 95.59120  |
| 25M    | MLA        | 4.64187 [4.63024, 4.65350] | 4.60447 [4.59376, 4.61517] | 103.75039      | 99.93967  |
| 25M    | MTP        | 4.58753 [4.57869, 4.59637] | 4.55093 [4.54195, 4.55992] | 98.25853       | 94.72761  |
| 25M    | MoE        | 4.57400 [4.56353, 4.58446] | 4.53808 [4.53074, 4.54542] | 96.93992       | 93.51539  |
| 25M    | MLA+MoE    | 4.61517 [4.60755, 4.62278] | 4.58044 [4.57065, 4.59022] | 101.00970      | 97.56534  |
| 25M    | V3 Routing | 4.57588 [4.57094, 4.58081] | 4.53894 [4.53562, 4.54227] | 97.11506       | 93.59262  |
| 50M    | Dense      | 4.41730 [4.41311, 4.42148] | 4.37893 [4.37419, 4.38366] | 82.87347       | 79.75383  |
| 50M    | MLA        | 4.46548 [4.45680, 4.47415] | 4.42797 [4.42035, 4.43559] | 86.96831       | 83.76537  |
| 50M    | MTP        | 4.39978 [4.39217, 4.40738] | 4.36401 [4.35711, 4.37090] | 81.43687       | 78.57476  |
| 50M    | MoE        | 4.38648 [4.38208, 4.39089] | 4.35185 [4.34620, 4.35749] | 80.35844       | 77.62375  |
| 50M    | MLA+MoE    | 4.43293 [4.42915, 4.43671] | 4.39658 [4.39114, 4.40203] | 84.17888       | 81.17524  |
| 50M    | V3 Routing | 4.39140 [4.38655, 4.39624] | 4.35484 [4.34966, 4.36002] | 80.75479       | 77.85631  |

*Note.* Perplexity is  $\exp(\text{loss})$  and is therefore a monotonic restatement of cross-entropy rather than an independent inferential endpoint. Every cell uses the identical fixed set of 400 non-overlapping, corpus-spanning windows per split.

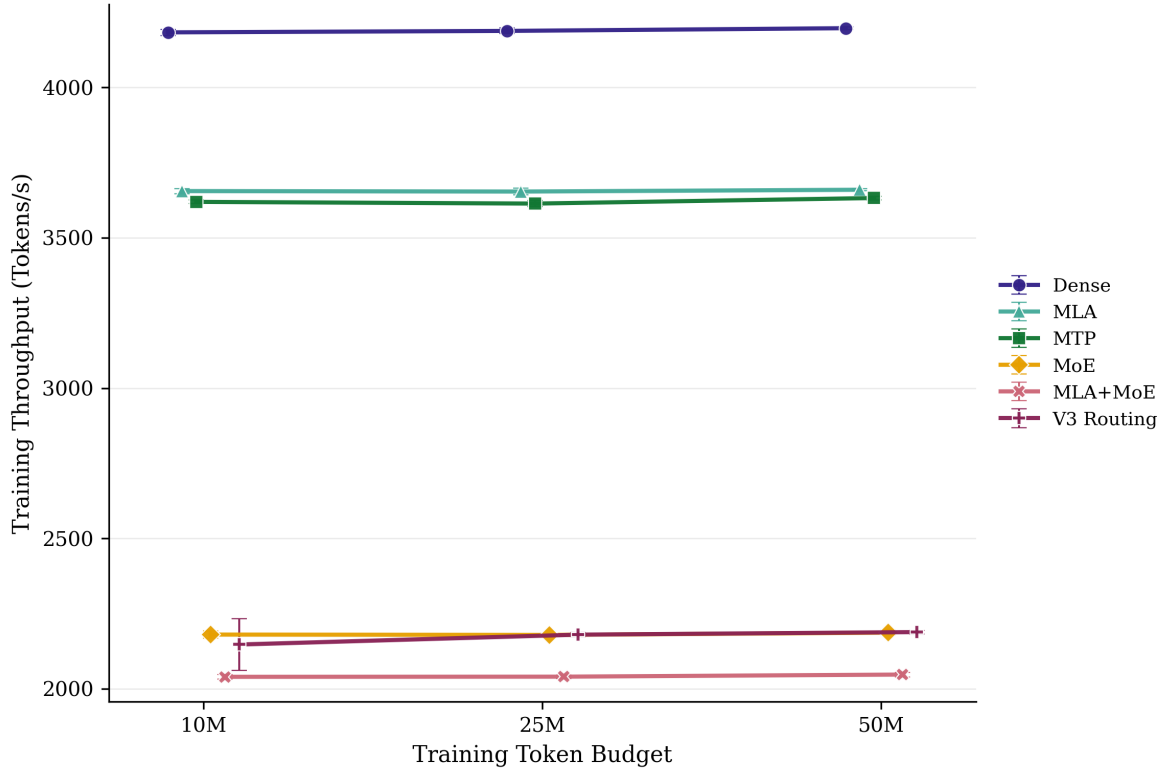


**Figure A.1:** Validation loss across token budgets. Points are means across ten aligned seeds and bars are 95% Student- $t$  intervals; lower is better. Small horizontal offsets separate models at the same nominal budget and have no statistical meaning. Every budget uses the identical fixed set of 400 non-overlapping, corpus-spanning validation windows.

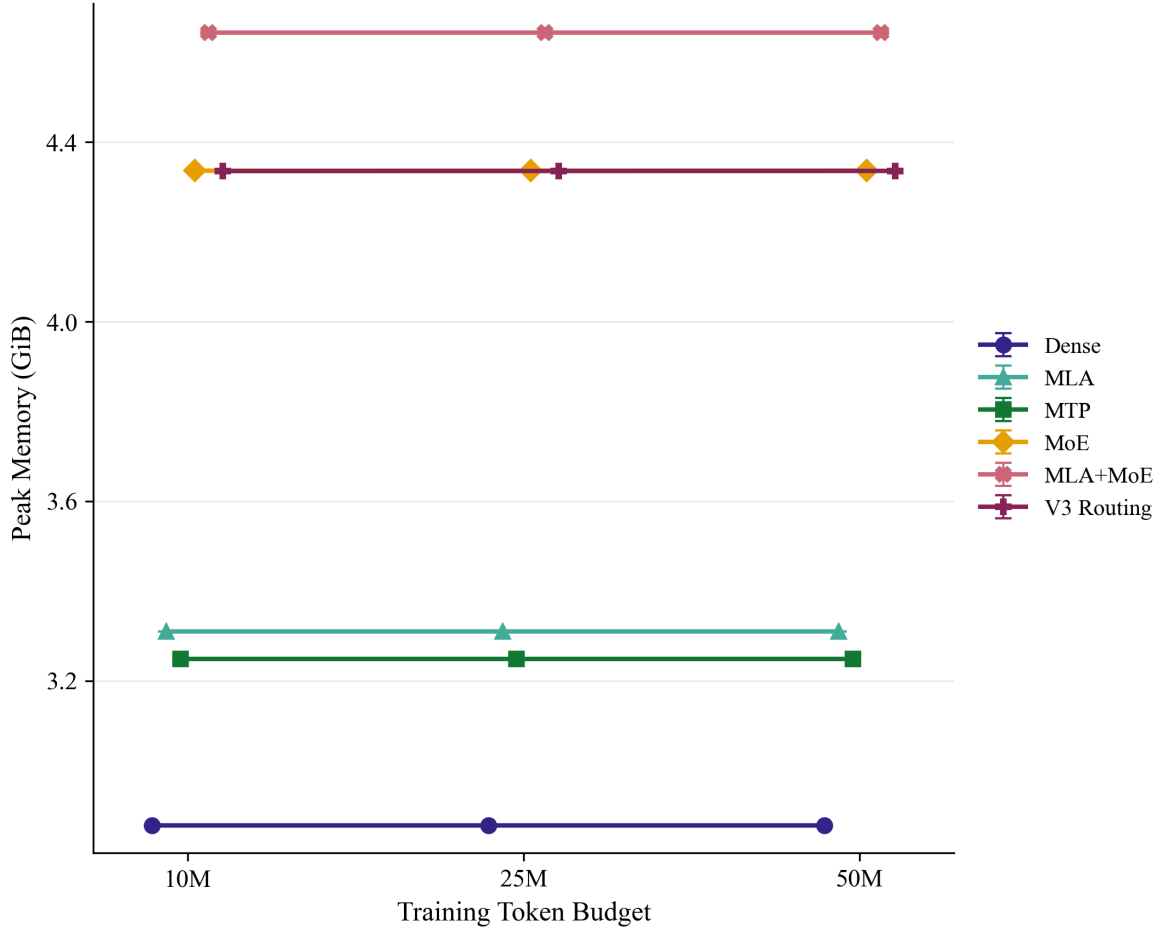
**Table A.2:** Full systems and parameter-exposure descriptives. Training-step throughput entries are means with 95% Student- $t$  intervals across ten aligned seeds. Training and evaluation peak-memory entries are means in GiB. Parameter counts and exposure values are deterministic at a fixed architecture and executed token budget.

| Budget | Model      | Step tokens/s [95% CI]  | Train GiB | Eval. GiB | Total M | Activated M | Tokens/total | Tokens/activated |
|--------|------------|-------------------------|-----------|-----------|---------|-------------|--------------|------------------|
| 10M    | Dense      | 4183.0 [4173.1, 4193.0] | 2.88      | 2.01      | 120.945 | 120.945     | 0.083        | 0.083            |
| 10M    | MLA        | 3655.3 [3646.7, 3663.9] | 3.31      | 2.30      | 137.460 | 137.460     | 0.073        | 0.073            |
| 10M    | MTP        | 3619.4 [3613.3, 3625.4] | 3.25      | 2.24      | 136.305 | 136.305     | 0.073        | 0.073            |
| 10M    | MoE        | 2180.4 [2169.3, 2191.6] | 4.34      | 3.54      | 220.146 | 78.589      | 0.045        | 0.127            |
| 10M    | MLA+MoE    | 2040.0 [2032.5, 2047.4] | 4.64      | 3.81      | 236.662 | 95.104      | 0.042        | 0.105            |
| 10M    | V3 Routing | 2147.3 [2062.2, 2232.5] | 4.34      | 3.54      | 220.146 | 78.589      | 0.045        | 0.127            |
| 25M    | Dense      | 4187.9 [4178.7, 4197.0] | 2.88      | 2.01      | 120.945 | 120.945     | 0.207        | 0.207            |
| 25M    | MLA        | 3653.7 [3642.4, 3665.0] | 3.31      | 2.30      | 137.460 | 137.460     | 0.182        | 0.182            |
| 25M    | MTP        | 3613.7 [3603.9, 3623.5] | 3.25      | 2.24      | 136.305 | 136.305     | 0.183        | 0.183            |
| 25M    | MoE        | 2179.3 [2174.1, 2184.4] | 4.34      | 3.54      | 220.146 | 78.589      | 0.114        | 0.318            |
| 25M    | MLA+MoE    | 2040.5 [2034.3, 2046.6] | 4.64      | 3.81      | 236.662 | 95.104      | 0.106        | 0.263            |
| 25M    | V3 Routing | 2180.2 [2173.6, 2186.7] | 4.34      | 3.54      | 220.146 | 78.589      | 0.114        | 0.318            |
| 50M    | Dense      | 4196.9 [4192.9, 4201.0] | 2.88      | 2.01      | 120.945 | 120.945     | 0.413        | 0.413            |
| 50M    | MLA        | 3659.7 [3656.3, 3663.0] | 3.31      | 2.30      | 137.460 | 137.460     | 0.364        | 0.364            |
| 50M    | MTP        | 3632.2 [3626.6, 3637.7] | 3.25      | 2.24      | 136.305 | 136.305     | 0.367        | 0.367            |
| 50M    | MoE        | 2186.8 [2178.2, 2195.4] | 4.34      | 3.54      | 220.146 | 78.589      | 0.227        | 0.636            |
| 50M    | MLA+MoE    | 2047.5 [2039.7, 2055.2] | 4.64      | 3.81      | 236.662 | 95.104      | 0.211        | 0.526            |
| 50M    | V3 Routing | 2188.7 [2182.6, 2194.8] | 4.34      | 3.54      | 220.146 | 78.589      | 0.227        | 0.636            |

*Note.* Step throughput uses synchronized optimizer-step time and excludes evaluation, logging callbacks, checkpoint serialization, and artifact writing. Training and evaluation peaks are measured separately with `torch.cuda.max_memory_allocated`. All parameters were trainable, so tokens per trainable parameter equal tokens per total parameter. Activated parameters are an analytical architecture-level estimate, not measured FLOPs.



**Figure A.2:** Synchronized training-step throughput across token budgets. Points are means across ten aligned seeds and bars are 95% Student- $t$  intervals; higher is faster. The timer includes batch transfer, forward and backward computation, gradient clipping, and the optimizer update while excluding evaluation, logging callbacks, checkpoint serialization, and artifact writing. Horizontal offsets are display-only.



**Figure A.3:** Peak training-phase CUDA allocation across token budgets. Points are means across ten aligned seeds and bars are 95% Student-*t* intervals. The metric is `torch.cuda.max_memory_allocated`; it does not represent reserved memory, total device occupancy, host memory, or the separately recorded evaluation peak. Horizontal offsets are display-only.

## B. Complete Inferential Results

The main text reports the comparisons needed to answer the research questions. This appendix provides the complete prespecified test-loss family, the paired systems contrasts, the within-model token-budget changes, and supplementary omnibus tests. The aligned seed is the repeated experimental unit throughout.

**Table B.1:** Complete prespecified test-loss contrasts. Differences are Model B minus Model A, so negative values favor Model B. Confidence intervals use 20,000 paired percentile-bootstrap resamples; exact two-sided sign-flip tests use the ten aligned seed differences. The final column applies the Holm adjustment across all 21 primary test-loss contrasts.

| Budget | Contrast (B – A) | Mean diff. | Bootstrap 95% CI | Cohen’s $d_z$ | B better | Exact $p$ | Holm $p$ |
|--------|------------------|------------|------------------|---------------|----------|-----------|----------|
| 10M    | MLA – Dense      | 0.058      | [0.040, 0.078]   | 1.760         | 0/10     | 0.001953  | 0.041016 |
| 10M    | MoE – Dense      | -0.054     | [-0.068, -0.040] | -2.270        | 10/10    | 0.001953  | 0.041016 |
| 10M    | MLA+MoE – Dense  | 0.000      | [-0.010, 0.010]  | 0.004         | 6/10     | 0.986328  | 1.000000 |
| 10M    | MLA+MoE – MLA    | -0.058     | [-0.084, -0.034] | -1.338        | 9/10     | 0.003906  | 0.041016 |
| 10M    | MLA+MoE – MoE    | 0.054      | [0.035, 0.073]   | 1.676         | 0/10     | 0.001953  | 0.041016 |
| 10M    | V3 Routing – MoE | 0.008      | [-0.003, 0.018]  | 0.444         | 1/10     | 0.208984  | 0.835938 |
| 10M    | MTP – Dense      | 0.000      | [-0.015, 0.016]  | 0.011         | 5/10     | 0.978516  | 1.000000 |
| 25M    | MLA – Dense      | 0.044      | [0.038, 0.052]   | 3.575         | 0/10     | 0.001953  | 0.041016 |
| 25M    | MoE – Dense      | -0.022     | [-0.032, -0.013] | -1.414        | 10/10    | 0.001953  | 0.041016 |
| 25M    | MLA+MoE – Dense  | 0.020      | [0.015, 0.026]   | 2.087         | 0/10     | 0.001953  | 0.041016 |
| 25M    | MLA+MoE – MLA    | -0.024     | [-0.035, -0.014] | -1.334        | 9/10     | 0.003906  | 0.041016 |
| 25M    | MLA+MoE – MoE    | 0.042      | [0.030, 0.054]   | 2.061         | 0/10     | 0.001953  | 0.041016 |
| 25M    | V3 Routing – MoE | 0.001      | [-0.006, 0.007]  | 0.079         | 4/10     | 0.822266  | 1.000000 |
| 25M    | MTP – Dense      | -0.009     | [-0.017, -0.002] | -0.714        | 8/10     | 0.044922  | 0.269531 |
| 50M    | MLA – Dense      | 0.049      | [0.043, 0.056]   | 4.603         | 0/10     | 0.001953  | 0.041016 |
| 50M    | MoE – Dense      | -0.027     | [-0.032, -0.022] | -3.402        | 10/10    | 0.001953  | 0.041016 |
| 50M    | MLA+MoE – Dense  | 0.018      | [0.016, 0.020]   | 5.056         | 0/10     | 0.001953  | 0.041016 |
| 50M    | MLA+MoE – MLA    | -0.031     | [-0.037, -0.026] | -3.420        | 10/10    | 0.001953  | 0.041016 |
| 50M    | MLA+MoE – MoE    | 0.045      | [0.039, 0.050]   | 4.836         | 0/10     | 0.001953  | 0.041016 |
| 50M    | V3 Routing – MoE | 0.003      | [-0.001, 0.007]  | 0.476         | 3/10     | 0.166016  | 0.830078 |
| 50M    | MTP – Dense      | -0.015     | [-0.022, -0.008] | -1.241        | 9/10     | 0.005859  | 0.041016 |

*Note.* Test loss is the primary inferential outcome. The all-budget Holm family controls multiplicity across the complete prespecified test-loss analysis while avoiding a redundant family based on perplexity, which is a monotonic transformation of loss.

**Table B.2:** Planned training-step throughput and peak-training-memory contrasts. Relative change is  $100(B - A)/A$ . Positive throughput changes favor Model B; negative memory changes favor Model B. The Holm adjustment is applied separately across the 21 planned contrasts for each systems metric.

| Budget | Contrast (B - A) | Throughput change | Throughput Holm $p$ | Memory change | Memory Holm $p$ |
|--------|------------------|-------------------|---------------------|---------------|-----------------|
| 10M    | MLA - Dense      | -12.6%            | < 0.000001          | +15.0%        | 0.041016        |
| 10M    | MoE - Dense      | -47.9%            | 0.033203            | +50.7%        | 0.041016        |
| 10M    | MLA+MoE - Dense  | -51.2%            | 0.033203            | +61.4%        | 0.041016        |
| 10M    | MLA+MoE - MLA    | -44.2%            | < 0.000001          | +40.3%        | 0.041016        |
| 10M    | MLA+MoE - MoE    | -6.4%             | 0.033203            | +7.1%         | 0.041016        |
| 10M    | V3 Routing - MoE | -1.5%             | 1.000000            | -0.02%        | 1.000000        |
| 10M    | MTP - Dense      | -13.5%            | 0.033203            | +12.9%        | 0.041016        |
| 25M    | MLA - Dense      | -12.8%            | 0.033203            | +15.0%        | 0.041016        |
| 25M    | MoE - Dense      | -48.0%            | 0.033203            | +50.7%        | 0.041016        |
| 25M    | MLA+MoE - Dense  | -51.3%            | 0.033203            | +61.4%        | 0.041016        |
| 25M    | MLA+MoE - MLA    | -44.2%            | < 0.000001          | +40.3%        | 0.041016        |
| 25M    | MLA+MoE - MoE    | -6.4%             | 0.033203            | +7.1%         | 0.041016        |
| 25M    | V3 Routing - MoE | +0.04%            | 1.000000            | -0.01%        | 1.000000        |
| 25M    | MTP - Dense      | -13.7%            | 0.033203            | +12.9%        | 0.041016        |
| 50M    | MLA - Dense      | -12.8%            | < 0.000001          | +15.0%        | 0.041016        |
| 50M    | MoE - Dense      | -47.9%            | 0.033203            | +50.7%        | 0.041016        |
| 50M    | MLA+MoE - Dense  | -51.2%            | 0.033203            | +61.4%        | 0.041016        |
| 50M    | MLA+MoE - MLA    | -44.1%            | 0.033203            | +40.3%        | 0.041016        |
| 50M    | MLA+MoE - MoE    | -6.4%             | 0.033203            | +7.1%         | 0.041016        |
| 50M    | V3 Routing - MoE | +0.09%            | 1.000000            | -0.01%        | 1.000000        |
| 50M    | MTP - Dense      | -13.5%            | 0.033203            | +12.9%        | 0.041016        |

*Note.* Systems contrasts describe this single-GPU implementation and synchronized training-step path. They do not estimate production kernel efficiency, distributed communication cost, inference latency, or KV-cache savings. Evaluation memory and active end-to-end throughput are retained separately in the canonical artifacts. For peak memory, very small but seed-consistent differences can yield small adjusted  $p$ -values; practical magnitude should therefore be read alongside statistical significance.

**Table B.3:** Test-loss change across token budgets. Differences are 50M minus 10M, so negative values indicate improvement. Confidence intervals are paired 95% Student- $t$  intervals across the ten aligned seeds; exact  $p$ -values use a two-sided sign-flip test.

| Model      | 10M     | 25M     | 50M     | 50M-10M | 95% CI           | Improved | Exact $p$ |
|------------|---------|---------|---------|---------|------------------|----------|-----------|
| Dense      | 4.99745 | 4.56003 | 4.37893 | -0.619  | [-0.629, -0.608] | 10/10    | 0.001953  |
| MLA        | 5.05513 | 4.60447 | 4.42797 | -0.627  | [-0.643, -0.612] | 10/10    | 0.001953  |
| MTP        | 4.99774 | 4.55093 | 4.36401 | -0.634  | [-0.650, -0.617] | 10/10    | 0.001953  |
| MoE        | 4.94357 | 4.53808 | 4.35185 | -0.592  | [-0.599, -0.584] | 10/10    | 0.001953  |
| MLA+MoE    | 4.99751 | 4.58044 | 4.39658 | -0.601  | [-0.615, -0.587] | 10/10    | 0.001953  |
| V3 Routing | 4.95151 | 4.53894 | 4.35484 | -0.597  | [-0.607, -0.586] | 10/10    | 0.001953  |

**Table B.4:** Supplementary global model-effect tests. Both procedures preserve the aligned-seed block structure; permutation  $p$ -values use 20,000 within-seed model-label permutations.

| Metric                   | Budget | RM-ANOVA $F(5, 45)$ | Perm. $p$ | Friedman $Q(5)$ | Perm. $p$ |
|--------------------------|--------|---------------------|-----------|-----------------|-----------|
| Validation loss          | 10M    | 34.709              | 0.000050  | 41.714          | 0.000050  |
| Validation loss          | 25M    | 52.206              | 0.000050  | 42.743          | 0.000050  |
| Validation loss          | 50M    | 175.134             | 0.000050  | 46.514          | 0.000050  |
| Training-step throughput | 10M    | 3761.399            | 0.000050  | 47.771          | 0.000050  |
| Training-step throughput | 25M    | 119787.359          | 0.000050  | 48.629          | 0.000050  |
| Training-step throughput | 50M    | 164603.928          | 0.000050  | 48.800          | 0.000050  |
| Peak training allocation | 10M    | 879395.622          | 0.000050  | 48.800          | 0.000050  |
| Peak training allocation | 25M    | 864038.591          | 0.000050  | 48.629          | 0.000050  |
| Peak training allocation | 50M    | 881398.631          | 0.000050  | 48.629          | 0.000050  |

*Note.* Perplexity tests are omitted because perplexity is a monotonic transformation of loss and therefore adds no independent ordering or inferential family.

## C. Mechanism Diagnostics

The routing and MTP measurements below are retained as mechanism-specific diagnostics rather than primary outcomes. Their temporal meaning is deliberately narrow. Routing statistics summarize the final evaluated test batch and MTP loss comes from the final optimizer step. They can support interpretation of the terminal model state but cannot establish full training-trajectory stability, expert specialization, or convergence speed.

**Table C.1:** Terminal routing diagnostics for the sparse models. Entries are means with 95% Student-*t* intervals across ten seeds.

| Budget | Model      | Aux. loss [95% CI]            | Routing entropy [95% CI]      | Load variance [95% CI]        |
|--------|------------|-------------------------------|-------------------------------|-------------------------------|
| 10M    | MoE        | 0.010058 [0.010052, 0.010065] | 1.572830 [1.563597, 1.582063] | 0.000062 [0.000057, 0.000068] |
| 10M    | MLA+MoE    | 0.010054 [0.010044, 0.010063] | 1.483566 [1.467581, 1.499550] | 0.000061 [0.000058, 0.000064] |
| 10M    | V3 Routing | 0.000000 [0.000000, 0.000000] | 1.306123 [1.293161, 1.319085] | 0.000076 [0.000065, 0.000087] |
| 25M    | MoE        | 0.010036 [0.010029, 0.010043] | 1.751765 [1.741705, 1.761825] | 0.000049 [0.000044, 0.000054] |
| 25M    | MLA+MoE    | 0.010035 [0.010030, 0.010040] | 1.645802 [1.633811, 1.657792] | 0.000049 [0.000045, 0.000053] |
| 25M    | V3 Routing | 0.000000 [0.000000, 0.000000] | 1.577958 [1.565377, 1.590540] | 0.000052 [0.000044, 0.000060] |
| 50M    | MoE        | 0.010030 [0.010024, 0.010037] | 1.968805 [1.957788, 1.979822] | 0.000048 [0.000042, 0.000053] |
| 50M    | MLA+MoE    | 0.010032 [0.010028, 0.010035] | 1.896455 [1.882605, 1.910305] | 0.000051 [0.000045, 0.000057] |
| 50M    | V3 Routing | 0.000000 [0.000000, 0.000000] | 1.859854 [1.847329, 1.872378] | 0.000054 [0.000045, 0.000063] |

*Note.* These are latest-forward snapshots from the final evaluated test pass, averaged across the 12 MoE layers and then summarized across seeds. They are not trajectory averages. V3 Routing’s auxiliary loss is exactly zero by design. The load-variance values are low in magnitude; the corresponding main-body figure uses a narrowed axis only to make those differences visible.

**Table C.2:** Final optimizer-step MTP auxiliary loss. The value is diagnostic and is not a training trajectory, convergence-rate estimate, or cross-model-comparable objective.

| Budget | Mean MTP loss | 95% CI             |
|--------|---------------|--------------------|
| 10M    | 6.75370       | [6.66048, 6.84691] |
| 25M    | 6.61215       | [6.51555, 6.70876] |
| 50M    | 6.61568       | [6.49465, 6.73670] |

## D. Reproducibility and Artifact Lineage

The report was audited against repository main at commit `2c01074fdfe81d30503f6bef00816f4146b5e04a`. All 180 run summaries record the clean training commit `b40a593fc1125c109c96fe18d07e594bc79c9ec9` and source-tree hash `d61ccfe51385d3d4287f6284941be194dfec864c0deb7d913be5f8f342287804`. The deterministic evidence index records 38 analysis artifacts, 22 figures, 180 run summaries, and 33 supporting scripts. Its SHA-256 hash at the audited repository state is `5f5ca42d1ca9c68df1eea816037758b3f9744e480597c06c52f2f983d1862835`.

### D.1 Canonical artifact chain

**Table D.1:** Canonical artifact lineage used to construct the report.

| Role                               | Canonical repository artifact                                                            |
|------------------------------------|------------------------------------------------------------------------------------------|
| Balanced experiment contract       | <code>results/analysis/balanced_10seed_matrix_manifest.csv</code>                        |
| Per-run machine-readable record    | <code>results/analysis/balanced_10seed_matrix_summary_flat.csv</code>                    |
| Descriptive statistics             | <code>results/analysis/balanced_10seed_matrix_descriptives_full_precision.csv</code>     |
| Global repeated-measures tests     | <code>results/analysis/balanced_10seed_matrix_global_tests_full_precision.csv</code>     |
| Planned paired contrasts           | <code>results/analysis/balanced_10seed_matrix_paired_contrasts_full_precision.csv</code> |
| Token-budget trends                | <code>results/analysis/balanced_10seed_matrix_budget_trends_full_precision.csv</code>    |
| Mechanism profiles                 | <code>results/analysis/balanced_10seed_matrix_mechanism_profiles.csv</code>              |
| Data and tokenizer provenance      | <code>results/analysis/final_data_tokenizer_provenance.json</code>                       |
| Hashes and evidence classification | <code>results/analysis/balanced_10seed_matrix_evidence_index.json</code>                 |

The statistical tables in this report were regenerated from the full-precision CSV artifacts and rounded only for presentation. The analysis scripts use 20,000 within-seed permutations for global tests and 20,000 paired bootstrap resamples for planned contrasts. Exact sign-flip probabilities are retained where their discrete resolution is relevant. The primary inferential family is the 21 planned test-loss contrasts across all budgets, adjusted by Holm’s procedure.

### D.2 Configuration and execution boundaries

The model and training configurations are represented by the tracked YAML files referenced by the manifest. The final corpus is FineWeb-Edu `sample-10BT` at pinned revision `87f09149ef4734204d70ed1d046ddc9ca3f2b8f9`, capped at 50,000 training, 1,000 validation, and 1,000 test documents. The retained provenance record identifies the streaming shuffle seed as 1337 and hashes the text, record manifests, token arrays, configurations, and tokenizer. The byte-level BPE tokenizer has vocabulary size 10,000, covers the byte alphabet, was trained only on retained training text, and adds no special tokens to the continuous language-model streams.

The runs used deterministic settings and exact atomic checkpoint resumption. A resumable run restores model, optimizer, random-number-generator, random-window sampler, timing, and metric state only

after its code, configuration, data, tokenizer, environment, and experiment identity match. These controls support exact continuation on the retained setup, but they do not guarantee bitwise identity across different machines, drivers, operating systems, accelerators, CUDA kernels, or library versions. Throughput and memory values are measurements of the retained local environment rather than portable hardware constants.

### **D.3 Regeneration and audit scope**

The repository contains dedicated scripts to build and audit the manifest, extract the flat run record, compute descriptives, conduct paired and global inference, construct token-budget trends and mechanism profiles, verify data/tokenizer provenance, produce all 22 canonical figures, and rebuild the evidence index. Completion, extraction, descriptive, global-test, paired-contrast, budget-trend, mechanism-profile, and evidence-lineage audits pass with no missing primary metrics or failed matrix cells. The completed repository reports 291 passing tests. Preflight validation checks all 180 identities, exact generated configurations, counterbalanced order, artifact hashes, tokenizer/model agreement, fixed held-out samples, and summary semantics.

The audit records establish internal artifact consistency; they do not remove the substantive external-validity limitations described in Section 5. In particular, the implementations remain small-scale, single-GPU analogues without distributed expert parallelism, specialized sparse kernels, FlashMLA, production cache systems, or frontier-scale sequential MTP modules.

### **D.4 Presentation policy**

Loss and perplexity are displayed to five decimal places in tables, effect sizes and parameter ratios to three,  $p$ -values to six, throughput to one decimal place, and memory to two decimal places in GiB. All calculations use full precision before presentation rounding. Included figures were copied only from paths classified by the evidence index.

## References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- [2] Damai Dai et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models, 2024.
- [3] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [4] DeepSeek-AI. DeepSeek-V3 technical report, 2024.
- [5] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.
- [6] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979. doi: 10.1214/aos/1176344552.
- [7] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [8] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction, 2024.
- [9] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022.
- [10] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [11] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.
- [12] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021.
- [13] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

- [14] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [15] Guilherme Penedo et al. The FineWeb datasets: Decanting the web for the finest text data at scale, 2024.
- [16] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding, 2021.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.